

第四章 语法分析

许畅

南京大学计算机学院

2026年春季

概要

- 语法分析器
- 上下文无关文法
- 语法分析技术
 - 自顶向下
 - 自底向上
- 语法分析器生成工具

语法分析器的作用

- 基本作用
 - 从词法分析器获得词法单元的序列，确认该序列是否可以由语言的文法生成
 - 对于语法错误的程序，报告错误信息
 - 对于语法正确的程序，生成**语法分析树** (简称**语法树**)
 - 通常并不真的产生这棵语法分析树

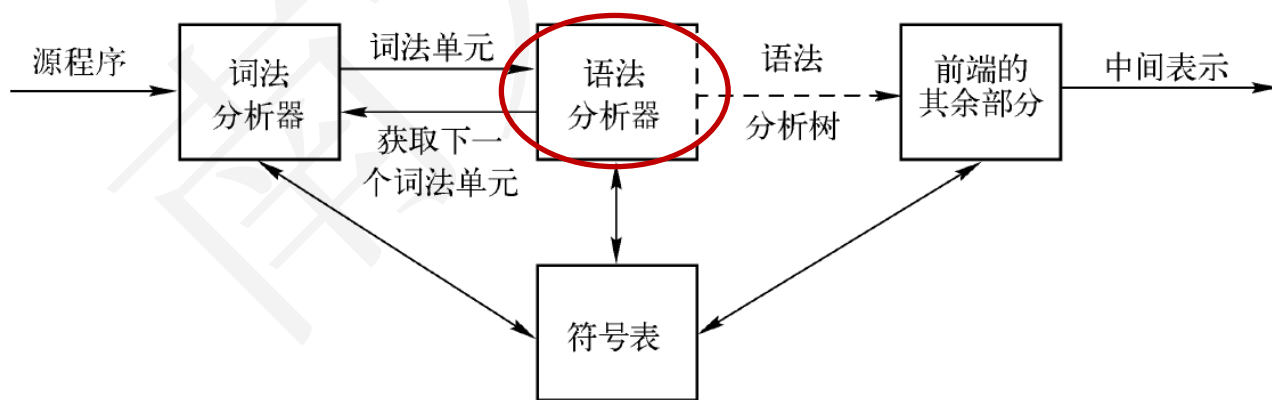


图 4-1 编译器模型中语法分析器的位置

语法分析器的分类

- 通用语法分析器
 - 可以对任意文法进行语法分析
 - 效率很低，不适合用于编译器
- 自顶向下语法分析器 (通常用于处理LL文法)
 - 从语法分析树的根部开始构造语法分析树
- 自底向上语法分析器 (通常用于处理LR文法)
 - 从语法分析树的叶子开始构造语法分析树
- 后两种方法
 - 总是从左到右、逐个扫描词法单元
 - 只能处理特定类型的文法，但足以描述程序设计语言

程序设计语言构造的描述

- 程序设计语言构造的语法可使用上下文无关文法 (CFG) 或BNF表示法来描述
 - 文法可给出精确易懂的语法规则
 - 可以自动构造出某些类型的文法的语法分析器
 - 文法指出了语言的**结构**，有助于进一步的语义处理/代码生成
 - 支持语言的演化和迭代

上下文无关文法

- 一个上下文无关文法 (CFG) 包含四个部分
 - 终结符号：组成串的基本符号 (词法单元名字)
 - 非终结符号：表示串的集合的语法变量
 - 在程序设计语言中通常对应于某个程序构造，比如 *stmt* (语句)
 - 开始符号：某个被指定的非终结符号
 - 它对应的串的集合就是文法的语言
 - 产生式：描述将终结符号和非终结符号组成串的方法
 - 形式：头 (左) 部 $\rightarrow$ 体 (右) 部
 - 头部是一个非终结符号，右部是一个符号串
 - 例子： $expression \rightarrow expression + term$

上下文无关文法的例子

- 简单算术表达式的文法

- 终结符号: $\text{id}, +, -, *, /, (,)$
- 非终结符号: $\text{expression}, \text{term}, \text{factor}$
- 开始符号: expression
- 产生式集合

$\text{expression} \rightarrow \text{expression} + \text{term}$

$\text{expression} \rightarrow \text{expression} - \text{term}$

$\text{expression} \rightarrow \text{term}$

$\text{term} \rightarrow \text{term} * \text{factor}$

$\text{term} \rightarrow \text{factor}$

$\text{factor} \rightarrow (\text{expression})$

$\text{term} \rightarrow \text{term} / \text{factor}$

$\text{factor} \rightarrow \text{id}$

文法简单形式的例子

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

- 注意
 - $|$ 是元符号 (即文法描述中的符号, 而不是文法符号)
 - 这里的 $($ 和 $)$ 不是元符号

推导 (1)

- 推导

- 将待处理的串中的某个非终结符号替换为这个非终结符号的某个产生式的体
- 从开始符号出发，不断进行上面的替换，就可以得到文法的不同句型

- 例子

- 文法： $E \rightarrow -E \mid E + E \mid E * E \mid (E) \mid id$
- 推导序列： $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(id)$

推导 (2)

- 推导的正式定义

- 如果 $A \rightarrow \gamma$ 是一个产生式, 那么 $\alpha A \beta \Rightarrow \alpha \gamma \beta$
- 最左/右推导 (leftmost/rightmost derivation): α/β 中 **不包含** 非终结符号
 - 符号: $\xRightarrow{*}_{lm} \quad \xRightarrow{*}_{rm}$

- 经过零步或者多步推导出: $\xRightarrow{*}$

- 对于任何串 $\alpha \xRightarrow{*} \alpha$
- 如果 $\alpha \xRightarrow{*} \beta$ 且 $\beta \Rightarrow \gamma$, 那么 $\alpha \xRightarrow{*} \gamma$

- 经过一步或者多步推导出: $\xRightarrow{+}$

- $\alpha \xRightarrow{+} \beta$ 即 $\alpha \xRightarrow{*} \beta$ 且 α 不等于 β (说法不严格)

句型/句子/语言

- 句型 (sentential form)
 - 如果 $S \xRightarrow{*} \alpha$, 那么 α 就是文法 S 的句型
 - 可能既包含非终结符号, 又包含终结符号, 也可以是空串
- 句子 (sentence)
 - 文法的句子就是不包含非终结符号的句型
- 语言
 - 文法 G 的语言就是 G 的句子的集合, 记为 $L(G)$
 - w 在 $L(G)$ 中当且仅当 w 是 G 的句子, 即 $S \xRightarrow{*} w$

语法分析树

- 推导的图形表示形式
 - 根结点的标号是文法的开始符号
 - 每个叶子结点的标号是非终结符号、终结符号或 ϵ
 - 每个内部结点的标号是非终结符号
 - 每个内部结点表示某个产生式的一次应用
 - 结点的标号为产生式头，其子结点从左到右是产生式的体
- 树的叶子组成的序列是根的文法符号的一个句型
- 一棵语法分析树可对应多个推导序列
 - 但只有唯一的最左推导及最右推导

例子

- 文法: $E \rightarrow -E \mid E + E \mid E * E \mid (E) \mid \text{id}$
- 句子: $-(\text{id} + \text{id})$

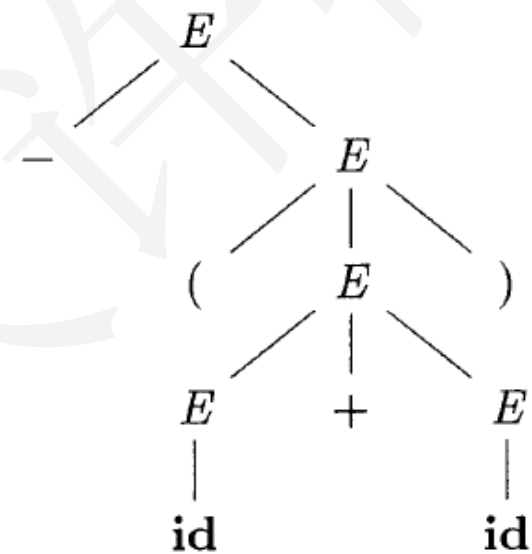


图 4-3 $-(\text{id} + \text{id})$ 的
语法分析树

从推导序列构造分析树

- 假设有推导序列
 - $A = a_1 \Rightarrow a_2 \Rightarrow \dots \Rightarrow a_n$
- 算法
 - 初始化: a_1 的分析树是标号为 A 的单个结点
 - 假设已经构造出 $a_{i-1} = X_1 X_2 \dots X_k$ 的分析树, 且 a_{i-1} 到 a_i 的推导是将 X_j 替换为 β , 那么在当前分析树中找出第 j 个非 ε 结点, 向这个结点增加构成 β 的子结点; 如果 $\beta = \varepsilon$, 则增加一个标号为 ε 的子结点

例子

- 推导序列

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(id + E) \Rightarrow -(id + id)$

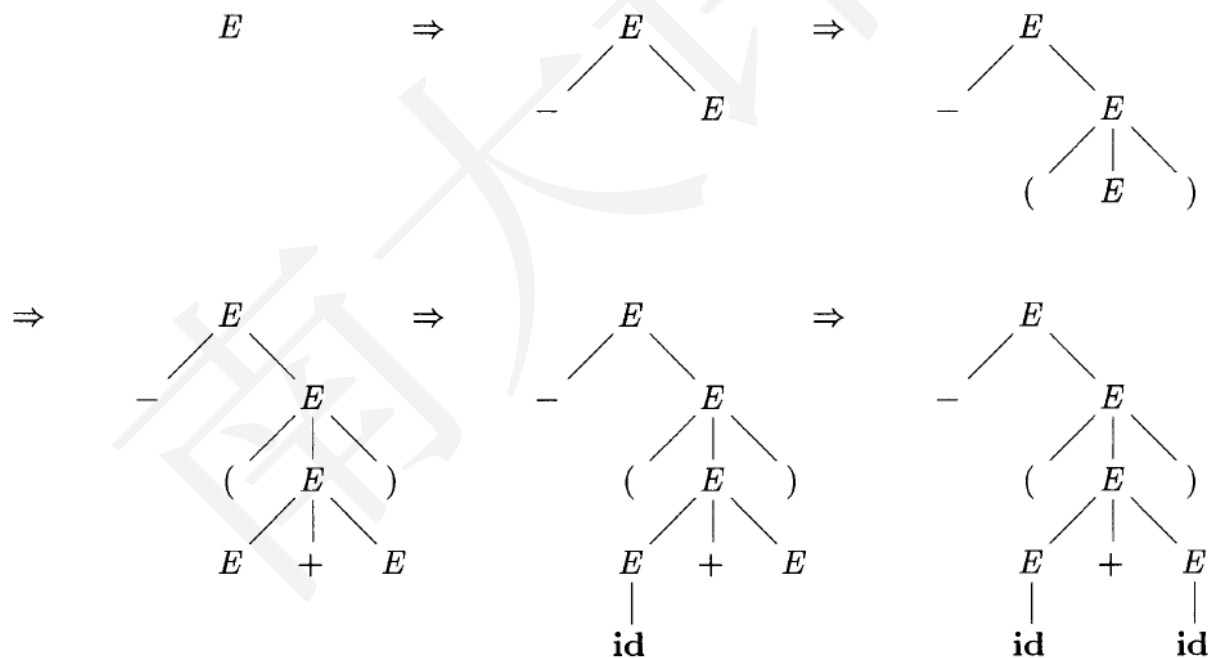


图 4-4 推导(4.8)的语法分析树序列

二义性 (1)

- 二义性 (ambiguity): 如果一个文法可以为某个句子生成多棵语法分析树, 这个文法就是二义的

- 例子

(1) $E \Rightarrow E + E \Rightarrow \text{id} + E \Rightarrow \text{id} + E * E \Rightarrow \text{id} + \text{id} * E$
 $\Rightarrow \text{id} + \text{id} * \text{id}$

(2) $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow \text{id} + E * E \Rightarrow \text{id} + \text{id} * E$
 $\Rightarrow \text{id} + \text{id} * \text{id}$

都是最左推导

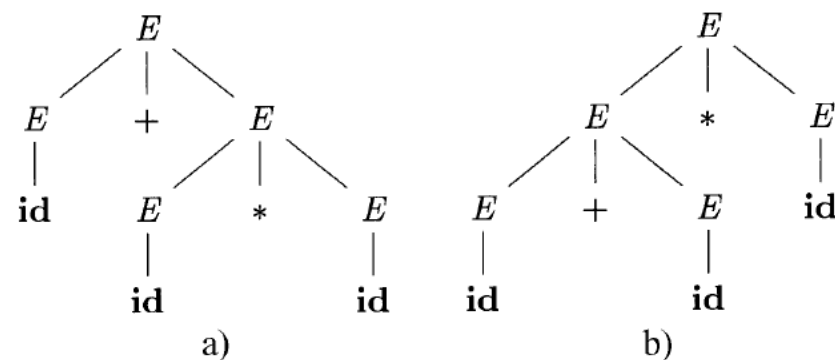


图 4-5 $\text{id} + \text{id} * \text{id}$ 的两棵语法树

二义性 (2)

- 程序设计语言的文法通常是**无二义**的
 - 否则就会导致一个程序有多种“正确”的解释
 - 如文法 $E \rightarrow -E \mid E + E \mid E * E \mid (E) \mid \text{id}$ 应对句子 $a + b * c$ 时
- 有些二义性的情况可以方便文法或语法分析器的设计 (后面介绍)
 - 但需要消二义性规则来剔除不要的语法分析树
 - 比如：先乘除后加减

词法分析和语法分析的比较

阶段	输入	输出	描述体系
词法分析	源程序符号串	词法单元序列	正则表达式
语法分析	词法单元序列	语法分析树	上下文无关文法

上下文无关文法和正则表达式 (1)

- 上下文无关文法比正则表达式的能力**更强**
 - 所有的正则语言都可以使用文法描述
 - 但有一些用文法描述的语言不能用正则表达式描述
- 证明
 - 首先 $S \rightarrow aSb \mid ab$ 描述了语言 $L \{ a^n b^n \mid n > 0 \}$ ，但这个语言无法用DFA识别
 - 假设有DFA识别此语言 L ，且这个DFA有 k 个状态。那么在识别 $a^{k+1}...$ 的输入串时，必然两次到达同一个状态。假设自动机在第 i 个和第 j 个 a 时进入同一个状态，那么：因为DFA识别 L ， $a^i b^j$ 必然到达接受状态，因此 $a^j b^i$ 必然也到达接受状态。
 - 直观地讲：有穷自动机**不能计数**

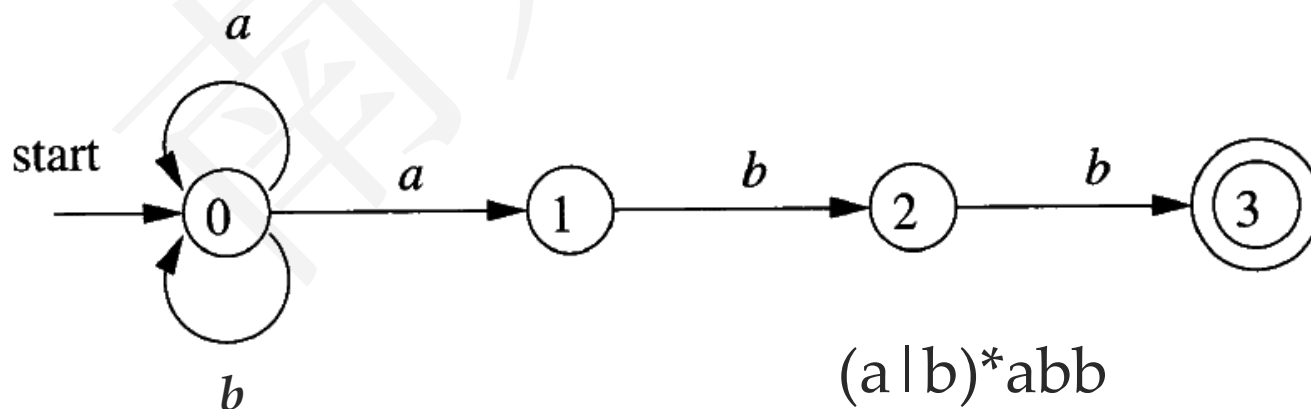
上下文无关文法和正则表达式 (2)

- 证明 (续)
 - 任何正则语言都可以表示为上下文无关文法的语言
 - 任何正则语言都必然有一个NFA，对于该NFA构造如下的上下文无关文法
 - 对NFA的每个状态 i ，创建非终结符号 A_i
 - 如果有 i 在输入 a 上到达 j 的转换，增加产生式 $A_i \rightarrow aA_j$
 - 如果 i 在输入 ϵ 上到达 j ，那么增加产生式 $A_i \rightarrow A_j$
 - 如果 i 是一个接受状态，增加产生式 $A_i \rightarrow \epsilon$
 - 如果 i 是开始状态，令 A_i 为所得文法的开始符号

NFA构造文法的例子

- $A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$
- $A_1 \rightarrow bA_2$
- $A_2 \rightarrow bA_3$
- $A_3 \rightarrow \varepsilon$

- 考虑baabb的推导和接受过程可知：NFA接受一个句子的运行过程实际就是该文法推导出该句子的过程



文法及其生成的语言

- 语言是从文法的开始符号出发，能推导得到的所有句子的集合
 - 文法 $G: S \rightarrow aS \mid a \mid b$, $L(G) = \{ a^i(a|b), i \geq 0 \}$
 - 文法 $G: S \rightarrow aSb \mid ab$, $L(G) = \{ a^n b^n, n \geq 1 \}$
 - 文法 $G: S \rightarrow (S)S \mid \varepsilon$, $L(G) = \{ \text{所有具有对称括号对的串} \}$
- 如何验证文法 G 所确定的语言 L
 - 证明 G 生成的每个串都在 L 中
 - 证明 L 中的每个串都能被 G 生成

设计文法 (1)

- 文法能够描述程序设计语言的大部分语法
 - 但不是全部，比如，标识符的先声明后使用则无法用上下文无关文法描述
 - 因此语法分析器接受的语言是程序设计语言的**超集**；必须通过语义分析来剔除一些符合文法、但不合法的程序

设计文法 (2)

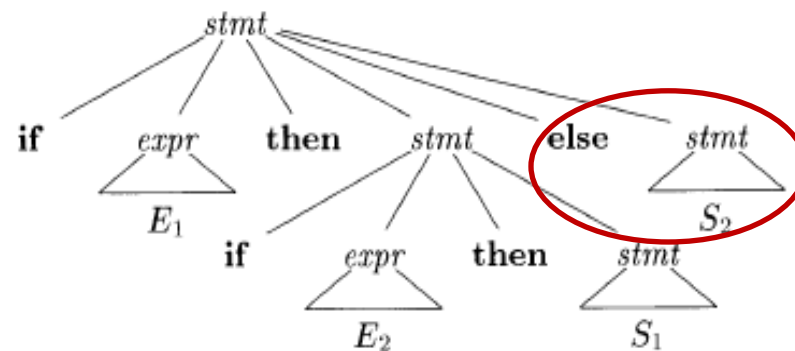
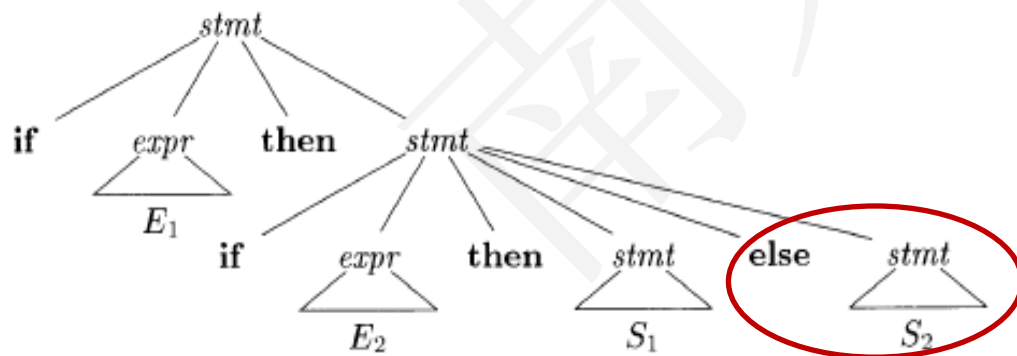
- 在进行高效的语法分析之前，需要对文法做以下处理
 - 消除二义性
 - 二义性：文法可以为一个句子生成多颗不同的分析树
 - 消除左递归
 - 左递归：文法中一个非终结符号 A 使得对某个串 α ，存在一个推导 $A \xRightarrow{+} A\alpha$ ，则称这个文法是左递归的
 - 提取左公因子

二义性的消除 (1)

- 一些二义性文法可被改成等价的无二义性的文法
- 例子

$stmt \rightarrow$ if $expr$ then $stmt$
| if $expr$ then $stmt$ else $stmt$
| other

- if E_1 then if E_2 then S_1 else S_2 的两棵语法树



二义性的消除 (2)

- 保证**else**和**最近未匹配**的**then**匹配
 - 要求在**then**和**else**之间出现的语句必须是匹配好的
- 引入 $matched_stmt$ 表示匹配好的语句
$$stmt \rightarrow matched_stmt \mid open_stmt$$
$$matched_stmt \rightarrow$$
$$\quad \text{if } expr \text{ then } matched_stmt \text{ else } matched_stmt$$
$$\quad \mid \text{ other}$$
$$open_stmt \rightarrow \text{if } expr \text{ then } stmt$$
$$\quad \mid \text{if } expr \text{ then } matched_stmt \text{ else } open_stmt$$
- 二义性的消除方法没有规律可循

左递归的消除

- 左递归 (left recursive) 的定义
 - 如果一个文法中有非终结符号 A 使得 $A \xRightarrow{+} A\alpha$, 那么这个文法就是左递归的
- 立即左递归
 - 文法中存在一个形如 $A \rightarrow A\alpha$ 的产生式
- 自顶向下的语法分析技术不能处理左递归的情况, 因此需要消除左递归, 但是自底向上的技术可以处理左递归

立即左递归的消除

- 假设非终结符号 A 存在立即左递归的情形

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

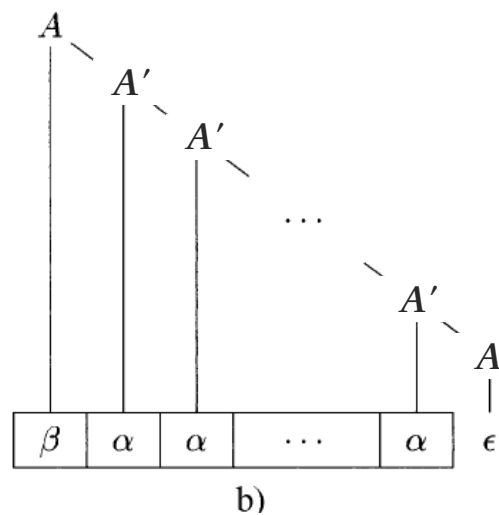
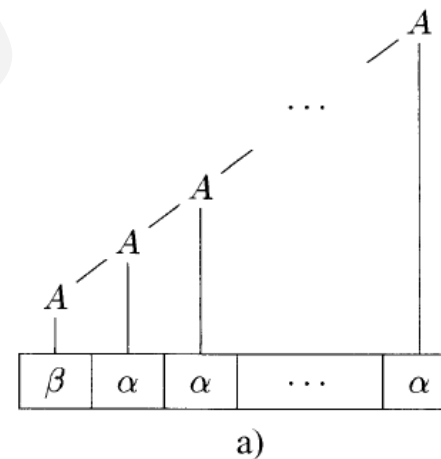
- 可替换为

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

- 观察：由 A 生成的串以某个 β_i 开头，然后跟上零个或多个 α_j

$A \rightarrow A\alpha \mid \beta$	$\Rightarrow$	$A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
------------------------------------	---------------	--



例子

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid \mathbf{id} \end{array} \quad \Rightarrow \quad \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \epsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \epsilon \\ F \rightarrow (E) \mid \mathbf{id} \end{array}$$

消除多步左递归

- 消除立即左递归的方法并不能消除因为多步推导而产生的左递归
 - 文法: $S \rightarrow Aa \mid b$ $A \rightarrow Ac \mid Sd \mid \epsilon$
 - $S \Rightarrow Aa \Rightarrow Sda$
- 如何消除?

通用的左递归消除方法

- 输入：没有环和 ϵ 产生式的文法 G
- 输出：等价的无左递归的文法
- 步骤

将文法的非终结符号排序为 $A_1, A_2, \dots, A_n$

for $i = 1$ to n do {

 for $j = 1$ to $i - 1$ do {

 将形如 $A_i \rightarrow A_j \gamma$ 的产生式替换为 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,

 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是以 A_j 为左部的所有产生式

 }

 消除 A_i 的立即左递归

}

通用的左递归消除的例子

- $S \rightarrow Aa \mid b$ $A \rightarrow Ac \mid Sd \mid \varepsilon$ 排列为 S, A
- $i = 1$ 时: 内层循环不运行, S 没有立即左递归
- $i = 2$ 时: $j = 1$, 处理规则 $A \rightarrow Sd$, 替换得到
 - $A \rightarrow Ac \mid \textcolor{red}{A}ad \mid \textcolor{red}{b}d \mid \varepsilon$
- 消除 A 的立即左递归
 - $S \rightarrow Aa \mid b$
 - $A \rightarrow bdA' \mid A'$
 - $A' \rightarrow cA' \mid adA' \mid \varepsilon$
- 通用左递归消除的问题在于很难找到新文法和旧文法推导之间的对应关系, 很难进行语义处理

本例子中的 ε 产生式恰好没
影响算法的正确性

预测分析法简介

- 试图从开始符号推导出输入符号串
- 每次为最左边的非终结符号选择适当的产生式
 - 通过查看下一个输入符号来选择这个产生式
 - 有多个可能的产生式时则无能为力
- 文法： $E \rightarrow + E E \mid - E E \mid \text{id}$ ，输入： $+ \text{id} - \text{id} \text{id}$
- 当两个产生式具有相同前缀时无法预测
 - 旧文法： $\text{stmt} \rightarrow \text{if expr then stmt else stmt}$
 - $\quad \quad \mid \text{if expr then stmt}$
 - 新文法： $\text{stmt} \rightarrow \text{if expr then stmt elsePart}$
 - $\quad \quad \text{elsePart} \rightarrow \text{else stmt} \mid \epsilon$

考虑 if a then ...

需提取左公因子

提取公因子的文法变换

- 算法
 - 输入：文法 G
 - 输出：等价的提取了左公因子的文法
 - 方法：对于每个非终结符号 A ，找出它的两个或者多个可选产生式体之间的最长公共前缀
 - $A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n \mid \gamma$
 - $A \rightarrow \alpha A' \mid \gamma \quad A' \rightarrow \beta_1 \mid \dots \mid \beta_n$
 - 其中 γ 是不以 α 开头的产生式体

例子

- 文法
 - $S \rightarrow iEtSeS \mid iEtS \mid a$
 - $E \rightarrow b$
- 对于 S 而言，最长的前缀是 $iEtS$ ，因此
 - $S \rightarrow iEtS S' \mid a$
 - $S' \rightarrow eS \mid \varepsilon$
 - $E \rightarrow b$

自顶向下的语法分析

- 为输入串构造语法分析树
 - 从分析树的根结点开始，按照**先根次序**，深度优先地创建各个结点
 - 对应于**最左推导**
- 基本步骤
 - 确定对句型中**最左边**的非终结符号应用哪个产生式
 - 然后对该产生式与输入符号进行匹配
- 关键问题
 - 确定对最左边的非终结符号应用**哪个**产生式

例子

- 文法

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

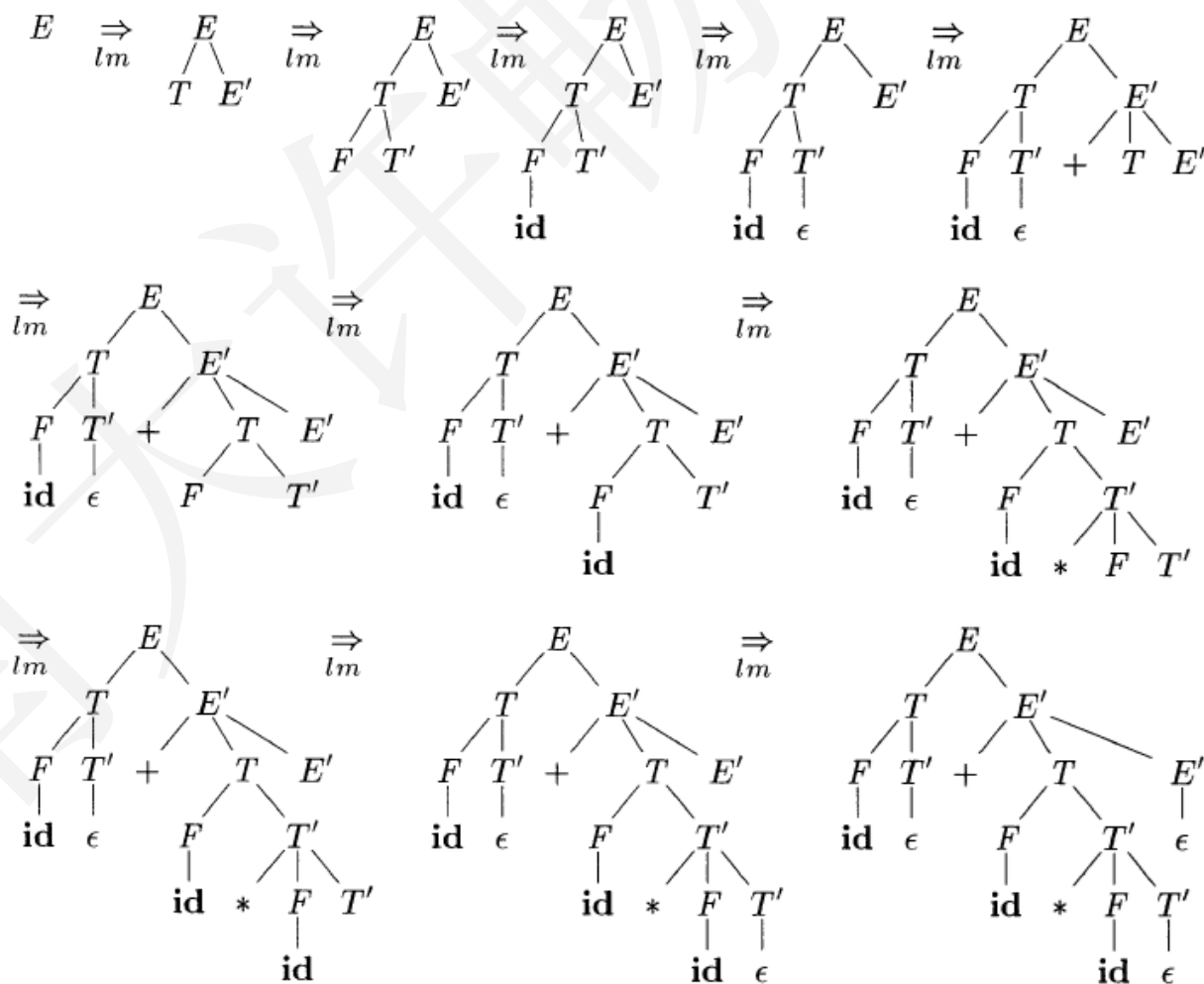
$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

- 输入

id + id * id

- 分析树见右



递归下降的语法分析

- 每个非终结符号对应于一个过程，该过程负责扫描此非终结符号对应的结构
- 程序执行从开始符号对应的过程开始
 - 当扫描整个输入串时宣布分析成功完成

```
void A() {  
1)      选择一个 A 产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
2)      for (  $i = 1$  to  $k$  ) {  
3)          if (  $X_i$  是一个非终结符号 )  
4)              调用过程  $X_i()$ ;  
5)          else if (  $X_i$  等于当前的输入符号  $a$  )  
6)              读入下一个输入符号;  
7)          else /* 发生了一个错误 */;  
      }  
}
```

递归下降分析技术的回溯

- 如果没有足够的信息来唯一确定可能的产生式，那么分析过程就产生回溯
 - 算法报告错误 (第7行; 第4行也可能返回错误) 并不意味着输入串不是句子, 也可能表示前面选错了产生式
 - 在第1行保存当前的扫描指针, 并把第7/4行改成
 - 回退到保存的指针, 并 GOTO 1) 去选择下一个产生式
 - 如果没有下一个产生式可选, 则报告错误
- 回溯需要来回扫描, 甚至撤销已完成的语义动作
- 解决方法 (后面介绍)
 - 设法通过一些信息来确定唯一可能的产生式

递归下降分析中回溯的例子

- 文法： $S \rightarrow cAd$ $A \rightarrow ab \mid a$ ； 输入串： cad
- 步骤
 - 调用函数 S ，选择唯一的产生式 $S \rightarrow cAd$
 - 输入中的 c 与句型中的 c 匹配，继续调用函数 A
 - A 首先选择产生式 $A \rightarrow ab$ ， a 与输入的 a 匹配，但 b 和输入的 d 不匹配
 - 回溯并选择下一个产生式 $A \rightarrow a$ ， a 与输入的 a 相匹配，对函数 A 的调用返回到 S 的调用
 - $S \rightarrow cAd$ 中最后的 d 与下一个输入 d 匹配，结束
- 因此 cad 是 S 的句子

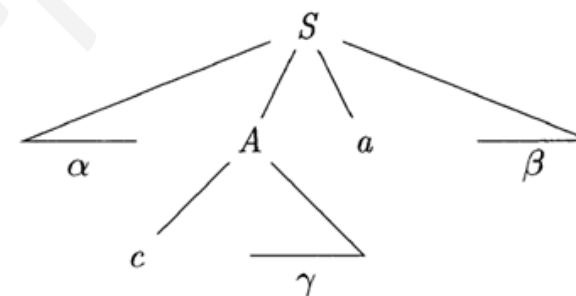
FIRST和FOLLOW

- 在自顶向下的分析技术中，使用向前看几个符号来确定产生式（通常只看一个符号）
- 当前句型是 $xA\beta$ ，而输入是 $xa \dots$ ，那么选择产生式 $A \rightarrow \alpha$ 的必要条件是下列之一
 - $\alpha \xRightarrow{*} a \dots$
 - $\alpha \xRightarrow{*} \epsilon$ ，且 β 以 a 开头，即在某个句型中 a 跟在 A 之后
 - 如果按照这两个条件选择时能够保证唯一性，那么我们就可以避免回溯
- 因此，我们定义FIRST和FOLLOW

FIRST

- FIRST(α)

- 可以从 α 推导得到串的首符号的集合
- 如果 $\alpha \xRightarrow{*} \varepsilon$, 那么 ε 也在FIRST(α)中



终结符号 c 在 FIRST(A)

- FIRST函数的意义

- A 的产生式 $A \rightarrow \alpha \mid \beta$, 且FIRST(α)和FIRST(β)不相交
- 下一个输入符号是 a , 若 $a \in \text{FIRST}(\alpha)$, 则选择 $A \rightarrow \alpha$, 若 $a \in \text{FIRST}(\beta)$, 则选择 $A \rightarrow \beta$

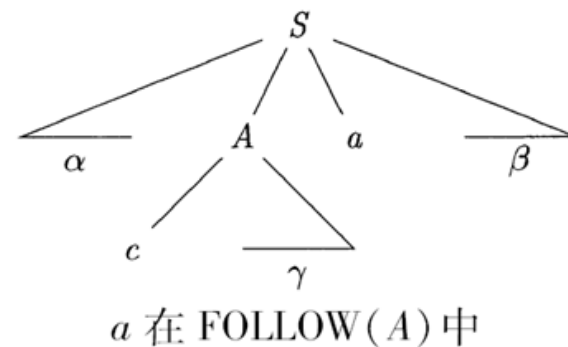
FIRST的计算方法

- 计算FIRST(X)
 - X 是终结符号，那么添加 X
 - X 是非终结符号，且 $X \rightarrow Y_1Y_2\dots Y_k$ 是产生式
 - 如果 a 在FIRST(Y_i)中，且 ϵ 在FIRST(Y_1), ..., FIRST(Y_{i-1})中，那么也添加 a
 - 如果 ϵ 在FIRST(Y_1), ..., FIRST(Y_k)中，那么也添加 ϵ
 - X 是非终结符号且 $X \rightarrow \epsilon$ ，那么也添加 ϵ
- 计算FIRST($X_1X_2\dots X_n$)
 - 添加FIRST(X_1)中所有非 ϵ 符号
 - 若 ϵ 在FIRST(X_1)中，添加FIRST(X_2)中所有非 ϵ 符号 ...
 - 若 ϵ 在所有FIRST(X_i)中，也添加 ϵ

FOLLOW

- FOLLOW(A)

- 可能在某些句型中紧跟在A右边的终结符号的集合
- 例如: $S \rightarrow \alpha A a \beta$, 终结符号 $a \in \text{FOLLOW}(A)$



- FOLLOW函数的意义

- 如果 $A \rightarrow \alpha$, 当 $\alpha \rightarrow \epsilon$ 或 $\alpha \Rightarrow \epsilon$ 时, $\text{FOLLOW}(A)$ 可以帮助我们选择恰当的的产生式
- 例如: $A \rightarrow \alpha$, 而 b 属于 $\text{FOLLOW}(A)$, 如果 $\alpha \Rightarrow \epsilon$, 而当前输入符号是 b , 则可以选择 $A \rightarrow \alpha$, 因为 A 最终到达了 ϵ , 而且后面跟着 b

FOLLOW的计算方法

- 算法
 - 将右端结束标记\$加入FOLLOW(S)中
 - 按照下面两个规则不断迭代，直到所有的FOLLOW集合都不再增长为止
 - 如果存在产生式 $A \rightarrow \alpha B \beta$ ，那么FIRST(β)中所有非 ϵ 的符号都加入FOLLOW(B)中
 - 如果存在一个产生式 $A \rightarrow \alpha B$ ，或者 $A \rightarrow \alpha B \beta$ 且FIRST(β)包含 ϵ ，那么FOLLOW(A)中所有符号都加入FOLLOW(B)中

例子 (1)

- 文法

- $E \rightarrow TE'$ $E' \rightarrow + TE' \mid \varepsilon$
- $T \rightarrow FT'$ $T' \rightarrow * FT' \mid \varepsilon$ $F \rightarrow (E) \mid \text{id}$

- FIRST集合的计算

- $\text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$
- $\text{FIRST}(E') = \{ +, \varepsilon \}$ $\text{FIRST}(T') = \{ *, \varepsilon \}$

- 由此可推出各个产生式右部的FIRST集合

- 如: $\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, \text{id} \}$

例子 (2)

- FOLLOW集合的计算
 - $\text{FOLLOW}(E) = \{ \$,) \}$ // E 是开始符号
 - $\text{FOLLOW}(E') = \{ \$,) \}$ // $E \rightarrow T E'$
 - $\text{FOLLOW}(T) = \{ +, \$,) \}$ // $E' \rightarrow + T E'$
 - $\text{FOLLOW}(T') = \{ +, \$,) \}$ // $T \rightarrow F T'$
 - $\text{FOLLOW}(F) = \{ *, +, \$,) \}$ // $T \rightarrow F T'$

LL(1)文法 (1)

- 定义：对文法的任意两个产生式 $A \rightarrow \alpha \mid \beta$
 - 不存在终结符号 a 使得 α 和 β 都可推导出以 a 开头的串
 - α 和 β 最多只有一个可推导出空串
 - 如果 β 可推导出空串，那么 α 不能推导出以 $\text{FOLLOW}(A)$ 中任何终结符号开头的串
- 等价于
 - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \Phi$ (条件一和二)
 - 如果 $\varepsilon \in \text{FIRST}(\beta)$ ，那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \Phi$ ；反之亦然 (条件三)

LL(1)文法 (2)

- 对于LL(1)文法，可以在自顶向下分析过程中，根据当前输入符号来确定使用的产生式
 - 产生式： $stmt \rightarrow \text{if}(\text{exp})\ stmt\ \text{else}\ stmt$
 - $\quad\quad\quad | \text{while}(\text{exp})\ stmt\ | \ a$
 - 输入： $\text{if}(\text{exp})\ \text{while}(\text{exp})\ a\ \text{else}\ a$
 - 首先选择产生式 $stmt \rightarrow \text{if}(\text{exp})\ stmt\ \text{else}\ stmt$ 得到句型 $\text{if}(\text{exp})\ stmt\ \text{else}\ stmt$
 - 然后把句型中的第一个 $stmt$ 展开，选择产生式 $stmt \rightarrow \text{while}(\text{exp})\ stmt$ ，得到句型 $\text{if}(\text{exp})\ \text{while}(\text{exp})\ stmt\ \text{else}\ stmt$
 - 再展开下个 $stmt$ ，得到 $\text{if}(\text{exp})\ \text{while}(\text{exp})\ a\ \text{else}\ stmt \dots$

预测分析表构造算法

- 输入：文法 G
- 输出：预测分析表 M
- 方法
 - 对于文法 G 的每个产生式 $A \rightarrow \alpha$
 - 对于 $\text{FIRST}(\alpha)$ 中的每个终结符号 a ，将 $A \rightarrow \alpha$ 加入到 $M[A, a]$ 中
 - 如果 ϵ 在 $\text{FIRST}(\alpha)$ ，那么对于 $\text{FOLLOW}(A)$ 中的每个符号 b ，将 $A \rightarrow \alpha$ 也加入到 $M[A, b]$ 中
 - 最后在所有的空白条目中填入**error**

例子

- 文法：

$$\begin{aligned}
 E &\rightarrow TE' & E' &\rightarrow +TE' \mid \varepsilon \\
 T &\rightarrow FT' & T' &\rightarrow *FT' \mid \varepsilon \\
 F &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

- FIRST集

- $E, T, F: \{ (, \text{id} \}$ $E': \{ +, \varepsilon \}$ $T': \{ *, \varepsilon \}$

- FOLLOW集

- $E, E': \{ \$,) \}$ $T, T': \{ +, \$,) \}$ $F: \{ *, +, \$,) \}$

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

预测分析表冲突的例子

- 文法： $S \rightarrow iEtSS' \mid a$ $S' \rightarrow eS \mid \epsilon$ $E \rightarrow b$
 - $\text{FIRST}(eS) = \{e\}$ ，使得 $S' \rightarrow eS$ 在 $M[S', e]$ 中
 - $\text{FOLLOW}(S') = \{\$, e\}$ ，使得 $S' \rightarrow \epsilon$ 也在 $M[S', e]$ 中
- LL(1)文法通常要求无二义性，而该文法存在二义性

非终结符号	输入符号					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

LL(1)文法的递归下降分析

- 递归下降语法分析程序由一组过程组成
- 每个非终结符号对应于一个过程，该过程负责扫描该非终结符号对应的结构
- 可以使用当前的输入符号来唯一地选择产生式

```
void A() {  
  1)  选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
  2)  for (  $i = 1$  to  $k$  ) {  
  3)      if (  $X_i$  是一个非终结符号 )  
  4)          调用过程  $X_i()$ ;  
  5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
  6)          读入下一个输入符号;  
  7)      else /* 发生了一个错误 */;  
  }  
}
```

如果当前输入符号为 a ，那么选择 $M[A, a]$ 中的产生式

非递归的预测分析 (1)

- 在自顶向下分析的过程中，我们总是
 - 匹配掉句型中左边的所有终结符号
 - 对于最左边的非终结符号，选择适当的产生式展开
 - 匹配成功的终结符号不会再被考虑，因此只需要记住句型的余下部分，以及尚未匹配的输入终结符号串
 - 由于展开的动作总是发生在余下部分的左端，我们可以用栈来存放这些符号

非递归的预测分析 (2)

- 分析时的处理过程
 - 初始化时，栈中仅包含开始符号 S (和 $\$$)
 - 如果栈顶元素是终结符号，那么进行匹配
 - 如果栈顶元素是非终结符号
 - 使用预测分析表来选择适当的产生式
 - 在栈顶用产生式右部替换产生式左部
- 对所有文法的预测分析都可以用同样的驱动程序

分析表驱动预测分析器

- 栈中符号序列为 α ， w 是已匹配的输入前缀， w' 是余下的输入，那么
 - S 推导出 $w\alpha$
 - 试图从 α 推导出余下的输入 w'
- 预测分析程序使用 $M[X, a]$ 来扩展 X ，将产生式的右部按倒序压入栈中

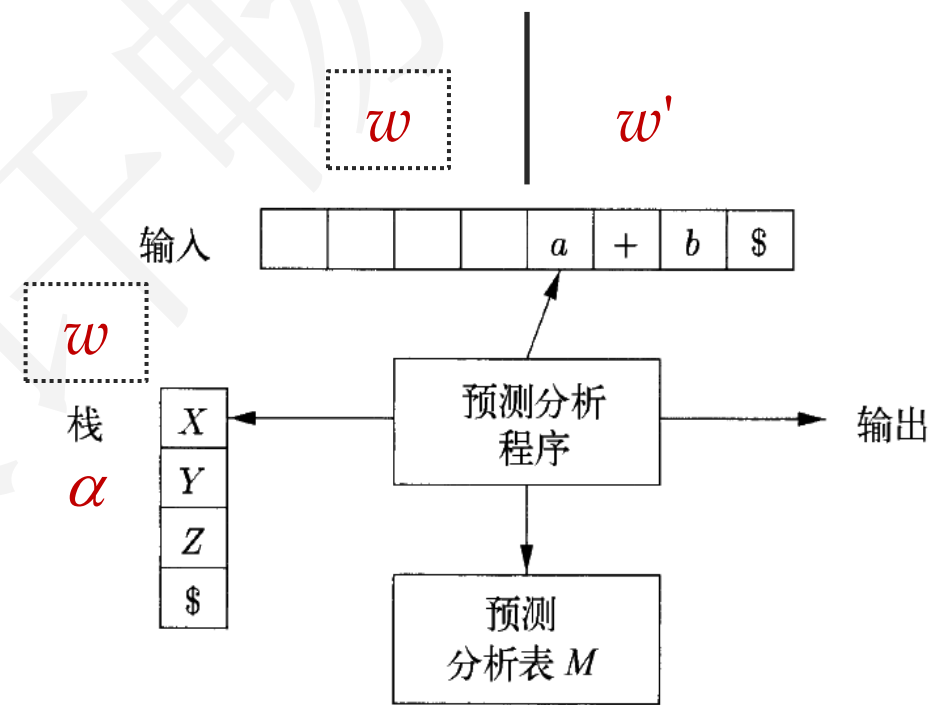


图 4-19 一个分析表驱动预测分析器的模型

预测分析算法

- 输入：串 w ，预测分析表 M
- 输出：如果 w 是句子，输出 w 的最左推导；否则报错
 - (1) 初始化：输入缓冲区中为 $w\$$ ，栈中为 $S\$$ ， ip 指向 w 的第一个符号
 - (2) 令 $X =$ 栈顶符号， ip 指向输入符号 a
 - if ($X == a$) X 出栈， ip 向前移动 // 与终结符号匹配成功
 - else if (X 是终结符号) $error()$ // 失配
 - else if ($M[X, a]$ 是报错条目) $error()$ // 无适当的产生式
 - else if ($M[X, a] = X \rightarrow Y_1Y_2...Y_k$) {
 - 输出产生式 $X \rightarrow Y_1Y_2...Y_k$
 - 弹出栈顶符号 X ，并将 $Y_k, Y_{k-1}, ..., Y_1$ 压入栈中
 - (3) 不断执行第二步，直到要么报错，要么栈中为空

分析表驱动预测分析的例子

- 输入：
id + id * id

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

已匹配	栈	输入	动作
	$E\$$	id + id * id\$	
	$TE'\$$	id + id * id\$	输出 $E \rightarrow TE'$
	$FT'E'\$$	id + id * id\$	输出 $T \rightarrow FT'$
	id $T'E'\$$	id + id * id\$	输出 $F \rightarrow \text{id}$
id	$T'E'\$$	+ id * id\$	匹配 id
id	$E'\$$	+ id * id\$	输出 $T' \rightarrow \epsilon$
id	+ $TE'\$$	+ id * id\$	输出 $E' \rightarrow +TE'$
id +	$TE'\$$	id * id\$	匹配 +

已匹配的部分加上栈中的符号必然是一个最左句型

自底向上的语法分析

- 为一个输入串构造语法分析树的过程
- 从叶子 (输入串中的终结符号, 将位于分析树的底端) 开始, 向上到达根结点
 - 在实际的语法分析过程中并不一定会构造出相应的分析树, 但是用分析树的概念可以方便理解
- 重要的自底向上语法分析的通用框架
 - 移入-归约 (shift-reduce)
- 简单LR技术 (SLR)、LR技术 (LR)

分析过程示例

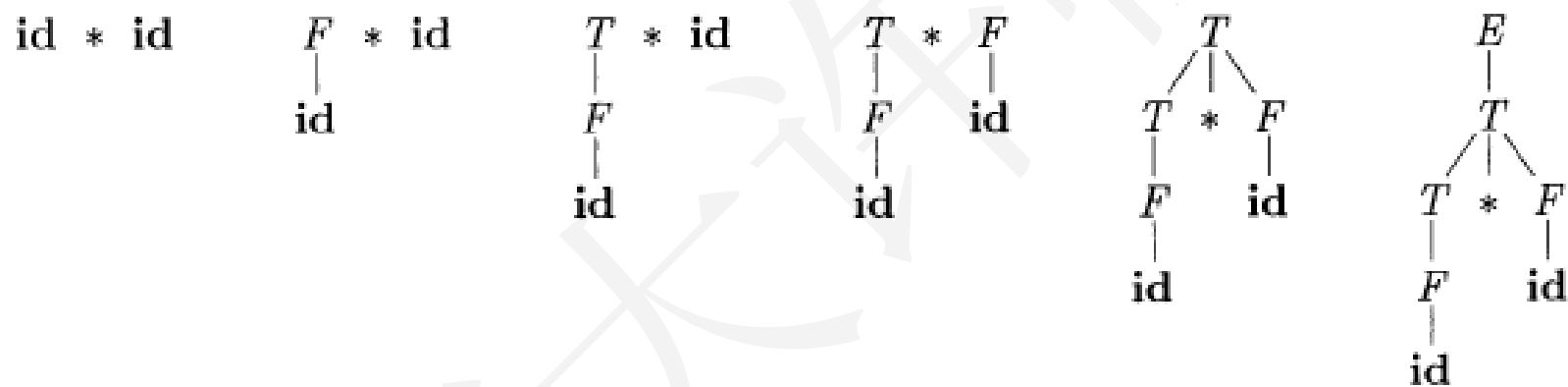


图 4-25 `id * id` 的自底向上分析过程

归约

- 自底向上的语法分析过程可以看成是从串 w 归约为文法开始符号 S 的过程
- 归约 (reduction) 步骤
 - 一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号
- 问题
 - 何时归约 (归约哪些符号串)?
 - 归约到哪个非终结符号?

例子

- $\text{id} * \text{id}$ 的归约过程
 - $\text{id} * \text{id}$, $F * \text{id}$, $T * \text{id}$, $T * F$, T , E
- 对于句型 $T * \text{id}$, 有两个子串和某产生式右部匹配
 - T 是 $E \rightarrow T$ 的右部
 - id 是 $F \rightarrow \text{id}$ 的右部
 - 为什么选择将 id 归约为 F , 而不是将 T 归约为 E ?
 - 原因: T 归约为 E 之后, $E * \text{id}$ 不再是句型
 - 问题: 如何确定这一点?

句柄

- 对输入从左到右扫描，并进行自底向上的语法分析，实际可以反向构造出一个最右推导
- 句柄 (handle)
 - 最右句型中和某个产生式体相匹配的子串，对它的归约代表了该最右句型的最右推导的最后一步
 - 正式定义：如果 $S \xRightarrow{*} \alpha A w \xRightarrow{} \alpha \beta w$ ，那么紧跟 α 之后的 β 是 $A \rightarrow \beta$ 的一个句柄
- 在一个最右句型中，句柄右边只有终结符号
- 如果文法是无二义性的，那么每个句型有且只有一个句柄

例子

- 输入: $\text{id} * \text{id}$

最右句型	句柄	归约用的产生式
$\text{id}_1 * \text{id}_2$	id_1	$F \rightarrow \text{id}$
$F * \text{id}_2$	F	$T \rightarrow F$
$T * \text{id}_2$	id_2	$F \rightarrow \text{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

移入-归约分析技术

- 使用一个栈来保存归约/扫描移入的文法符号
- 栈中符号 (从底向上) 和待扫描的符号组成了一个最右句型
- 开始时刻：栈中只包含 $\$$ ，而输入为 $w\$$
- 结束时刻：栈中为 $S\$$ ，而输入为 $\$$
- 在分析过程中，不断移入符号，并在识别到句柄时进行归约
- 句柄被识别时总是出现在栈的顶部 (后面解释)

主要分析动作

- 移入 (shift): 将下一个输入符号移入到栈顶
- 归约 (reduce): 将句柄归约为相应的非终结符号
 - 句柄总是在栈顶
 - 具体操作时弹出句柄, 压入被归约到的非终结符号
- 接受 (accept): 宣布分析过程成功完成
- 报错 (error): 发现语法错误, 调用错误恢复子程序

例子

栈	输入	动作
\$	$\text{id}_1 * \text{id}_2 \$$	移入
\$ id_1	$* \text{id}_2 \$$	按照 $F \rightarrow \text{id}$ 归约
\$ F	$* \text{id}_2 \$$	按照 $T \rightarrow F$ 归约
\$ T	$* \text{id}_2 \$$	移入
\$ $T *$	$\text{id}_2 \$$	移入
\$ $T * \text{id}_2$	\$	按照 $F \rightarrow \text{id}$ 归约
\$ $T * F$	\$	按照 $T \rightarrow T * F$ 归约
\$ T	\$	按照 $E \rightarrow T$ 归约
\$ E	\$	accept

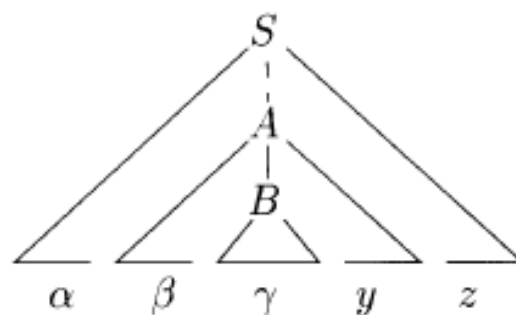
图 4-28 一个移入 - 归约语法分析器
在处理输入 $\text{id}_1 * \text{id}_2$ 时经历的格局

为什么句柄总是在栈顶?

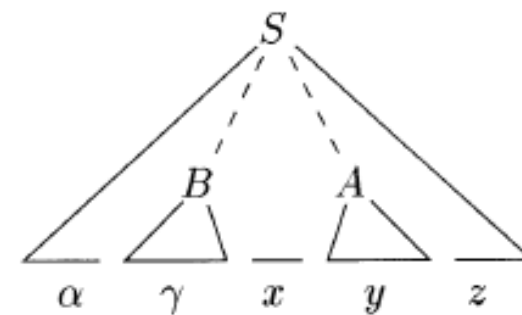
- 为什么每次归约得到的新句型的句柄仍在栈顶?
- 考虑最右推导的两个连续步骤的两种情况
 - 情况1): A 被替换为 βBy , 然后产生式体中的最右非终结符号 B 被替换为 γ (归约之后句柄为 βBy)
 - 情况2): A 首先被展开, 产生式体中只包含终结符号, 下一个最右非终结符号 B 位于 y 左侧

$$1) S \xRightarrow{rm} \alpha Az \xRightarrow{rm} \alpha \beta Byz \xRightarrow{rm} \alpha \beta \gamma yz$$

$$2) S \xRightarrow{rm} \alpha BxAz \xRightarrow{rm} \alpha Bxyz \xRightarrow{rm} \alpha \gamma xyz$$



情况 (1)



情况 (2)

移入-归约分析中的冲突

- 对于有些不能使用移入-归约分析的文法，不管用什么样的移入-归约分析器都会到达这样的格局
 - 即使知道了栈中所有内容、以及下面 k 个输入符号，人们仍然无法知道是否该进行归约 (移入-归约冲突)，或者不知道按照什么产生式进行归约 (归约-归约冲突)
 - 设栈中符号串是 $\alpha\beta$ ，接下来的 k 个符号是 x ，产生移入/归约冲突的原因是存在 y 和 y' 使得 $\alpha\beta xy$ 是最右句型且 β 是句柄 (需归约)，而 $\alpha\beta xy'$ 也是最右句型，但是句柄还在右边 (需移入)

移入-归约冲突的例子

$stmt \rightarrow$ **if** $expr$ **then** $stmt$
 | **if** $expr$ **then** $stmt$ **else** $stmt$
 | **other**

栈

$\dots$ **if** $expr$ **then** $stmt$

输入

else $\dots$ $\$$

归约-归约冲突的例子

- 输入为 $\text{id}(\text{id}, \text{id})$
- 冲突时的格局
 - 栈: $\dots \text{id}(\text{id}$ 输入: $, \text{id}) \dots$

(1)	<i>stmt</i>	→	<i>id</i> (<i>parameter_list</i>)
(2)	<i>stmt</i>	→	<i>expr</i> := <i>expr</i>
(3)	<i>parameter_list</i>	→	<i>parameter_list</i> , <i>parameter</i>
(4)	<i>parameter_list</i>	→	<i>parameter</i>
(5)	<i>parameter</i>	→	<i>id</i>
(6)	<i>expr</i>	→	<i>id</i> (<i>expr_list</i>)
(7)	<i>expr</i>	→	<i>id</i>
(8)	<i>expr_list</i>	→	<i>expr_list</i> , <i>expr</i>
(9)	<i>expr_list</i>	→	<i>expr</i>

图 4-30 有关过程调用和数组引用的产生式

LR语法分析技术

- LR(k)的语法分析概念
 - L表示从左到右扫描输入，R表示构造最右推导的逆过程
 - k 表示最多向前看 k 个符号
- 当 k 增大时，相应的语法分析器的规模急剧增大
 - $k=2$ 时，程序语言的语法分析器的规模通常非常庞大
 - 当 $k=0, 1$ 时，已经可以解决很多语法分析问题，因此具有实践意义
 - 我们只考虑 $k \leq 1$ 的情况

LR语法分析器的优点

- 由表格驱动，虽然手工构造表格工作量很大，但表格可以自动生成
- 对于几乎所有的程序设计语言，只要写出上下文无关文法，就能够构造出识别该语言的LR语法分析器
- 最通用的无回溯移入-归约分析技术
- 能分析的文法比LL(k)文法更多
 - 真超集

LR(0)项

- 项 (item): 文法的一个产生式加上在其中某处的一个点
 - $A \rightarrow \cdot XYZ$, $A \rightarrow X \cdot YZ$, $A \rightarrow XY \cdot Z$, $A \rightarrow XYZ \cdot$
 - 注意: $A \rightarrow \varepsilon$ 只对应一个项 $A \rightarrow \cdot$
- 直观含义
 - 项 $A \rightarrow \alpha \cdot \beta$ 表示已经扫描/归约到了 α , 并期望在接下来的输入中经过扫描/归约得到 β , 然后把 $\alpha\beta$ 归约到 A
 - 如果 β 为空, 表示我们~~可以~~把 α 归约为 A
- 项也可以用一对整数表示: (i, j) 表示第 i 条产生式, 点位于右部第 j 个位置

LR(0)项集规范族的构造

- 三个相关定义
 - 增广文法
 - 项集闭包CLOSURE
 - GOTO函数
- 增广文法 (augmented grammar)
 - G 的增广文法 G' 是在 G 中增加新开始符号 S' ，并加入产生式 $S' \rightarrow S$ 而得到的
 - 显然 G' 和 G 接受相同的语言，且按照 $S' \rightarrow S$ 进行归约实际上就表示已经将输入符号串归约成为开始符号

项集闭包CLOSURE

- 项集闭包 (CLOSURE): 如果 I 是文法 G 的一个项集, $\text{CLOSURE}(I)$ 就是根据下列两条规则从 I 构造得到的项集
 - 将 I 中的各项加入 $\text{CLOSURE}(I)$ 中
 - 如果 $A \rightarrow \alpha \cdot B \beta$ 在 $\text{CLOSURE}(I)$ 中, 而 $B \rightarrow \gamma$ 是一个产生式, 且项 $B \rightarrow \cdot \gamma$ 不在 $\text{CLOSURE}(I)$ 中, 就将该项加入其中, 不断应用该规则直到没有新项可加入
- 意义
 - $A \rightarrow \alpha \cdot B \beta$, 表示希望看到由 $B\beta$ 推导出的串, 那要先看到由 B 推导出的串, 因此加上 B 的各个产生式对应的项

CLOSURE(I)的构造算法

```
SetOfItems CLOSURE( $I$ ) {  
     $J = I$ ;  
    repeat  
        for ( $J$ 中的每个项  $A \rightarrow \alpha \cdot B \beta$ )  
            for ( $G$  的每个产生式  $B \rightarrow \gamma$ )  
                if (项  $B \rightarrow \cdot \gamma$  不在  $J$  中)  
                    将  $B \rightarrow \cdot \gamma$  加入  $J$  中;  
    until 在某一轮中没有新的项被加入到  $J$  中;  
    return  $J$ ;  
}
```

例子

- 增广文法
 - $E' \rightarrow E \quad E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$
 - $F \rightarrow (E) \mid \text{id}$
- 项集{ $[E' \rightarrow \cdot E]$ }的闭包
 - $[E' \rightarrow \cdot E]$ 在闭包中
 - $[E \rightarrow \cdot E + T], [E \rightarrow \cdot T]$ 在闭包中
 - $[T \rightarrow \cdot T * F], [T \rightarrow \cdot F]$ 在闭包中
 - $[F \rightarrow \cdot (E)], [F \rightarrow \cdot \text{id}]$ 在闭包中

GOTO函数

- GOTO函数

- I 是一个项集， X 是一个文法符号，则 $\text{GOTO}(I, X)$ 定义为 I 中所有形如 $[A \rightarrow \alpha \cdot X \beta]$ 的项所对应的项 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包

- 例如

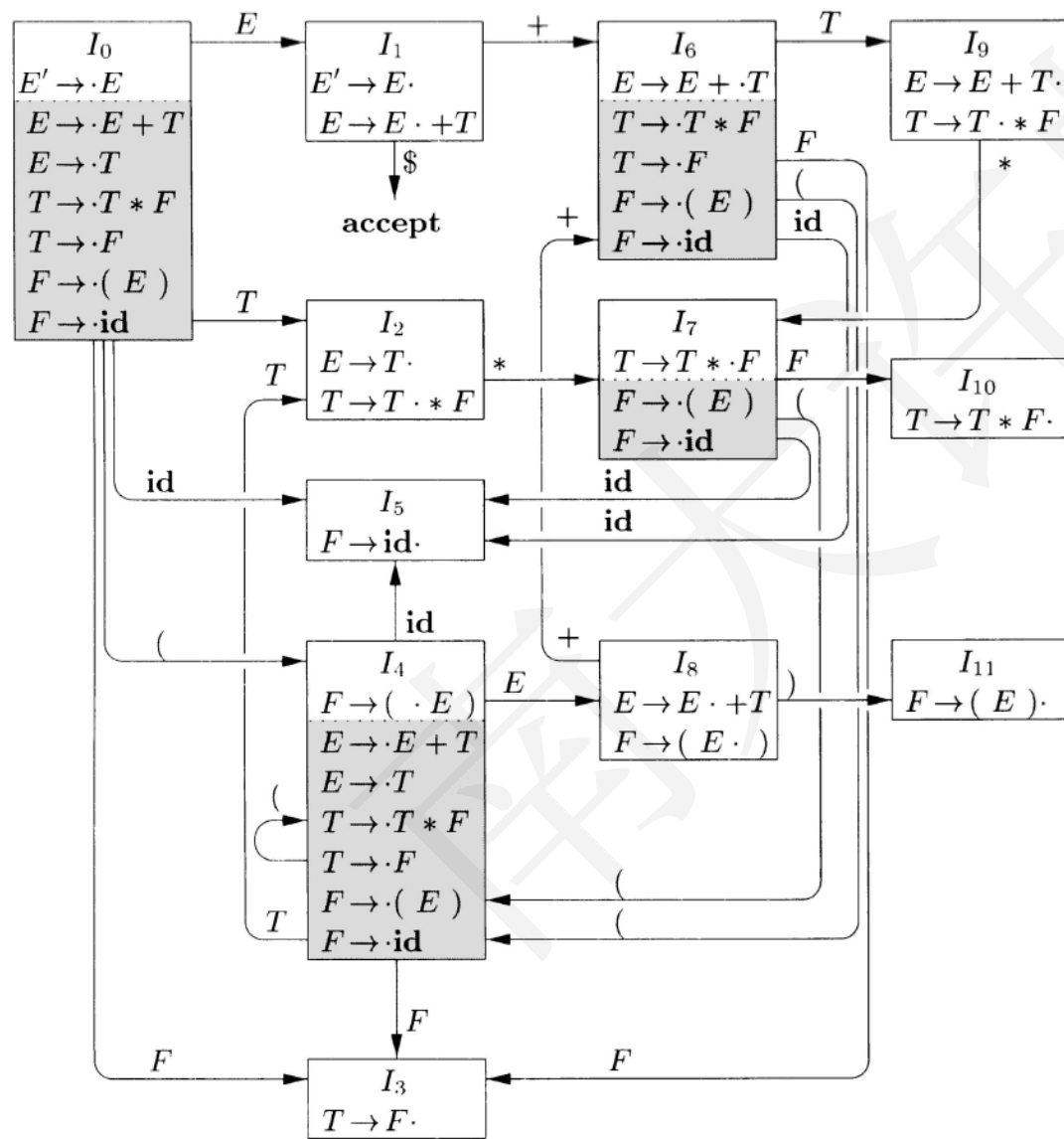
- $I = \{ [E' \rightarrow E \cdot], [E \rightarrow E \cdot + T] \}$
- $\text{GOTO}(I, +)$ 计算如下
 - I 中只有一个项的点后面跟着 $+$ ，对应的项为 $[E \rightarrow E + \cdot T]$
 - $\text{CLOSURE}(\{ [E \rightarrow E + \cdot T] \}) =$
 $\{ [E \rightarrow E + \cdot T], [T \rightarrow \cdot T^* F], [T \rightarrow \cdot F], [F \rightarrow \cdot (E)], [F \rightarrow \cdot \text{id}] \}$

求LR(0)项集规范族的算法

- 从初始项集开始，不断计算各种可能的后继，直到生成所有的项集

```
void items( $G'$ ) {  
     $C = \{ \text{CLOSURE}(\{[S' \rightarrow \cdot S]\}) \};$   
    repeat  
        for ( $C$  中的每个项集  $I$ )  
            for (每个文法符号  $X$ )  
                if (  $\text{GOTO}(I, X)$  非空且不在  $C$  中 )  
                    将  $\text{GOTO}(I, X)$  加入  $C$  中;  
    until 在某一轮中没有新的项集被加入到  $C$  中;  
}
```


项集规范族构造示例



(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR(0)自动机的构造

- 构造方法
 - 基于规范LR(0)项集族可以构造LR(0)自动机
 - 规范LR(0)项集族中的每个项集对应于LR(0)自动机的一个状态
 - 状态转换：如果 $\text{GOTO}(I, X) = J$ ，则从 I 到 J 有一个标号为 X 的转换
 - 开始状态为 $\text{CLOSURE}(\{ [S' \rightarrow \cdot S] \})$ 对应的项集

LR(0)自动机的作用 (1)

- 假设文法符号串 γ 使LR(0)自动机从开始状态运行到状态 (项集) j
- 如果 j 中存在项 $A \rightarrow \alpha \cdot$, 那么
 - 在 γ 之后添加一些终结符号可以得到一个最右句型
 - α 是 γ 的后缀, 且是该句型的句柄 (对应于产生式 $A \rightarrow \alpha$)
 - 表示可能找到了当前最右句型的句柄, 可以归约
- 如果 j 中存在项 $B \rightarrow \alpha \cdot X\beta$, 那么
 - 在 γ 之后添加 $X\beta$ 和一些终结符号可以得到一个最右句型
 - 该句型中 $\alpha X\beta$ 是句柄, 但还没找到, 还需移入

LR(0)自动机的作用 (2)

- LR(0)自动机的使用
 - 移入-归约时，LR(0)自动机被用于识别句型
 - 已得到的文法符号序列对应于LR(0)自动机的一条路径
- 无需每次用该文法符号序列来运行LR(0)自动机
 - 文法符号可省略，由LR(0)状态可确定相应的文法符号
 - 在移入后，根据原来的栈顶状态可以知道新的状态
 - 在归约时，根据归约产生式的右部长度弹出相应状态，也可以根据此时的栈顶状态知道新的状态

LR(0)自动机的作用的演示

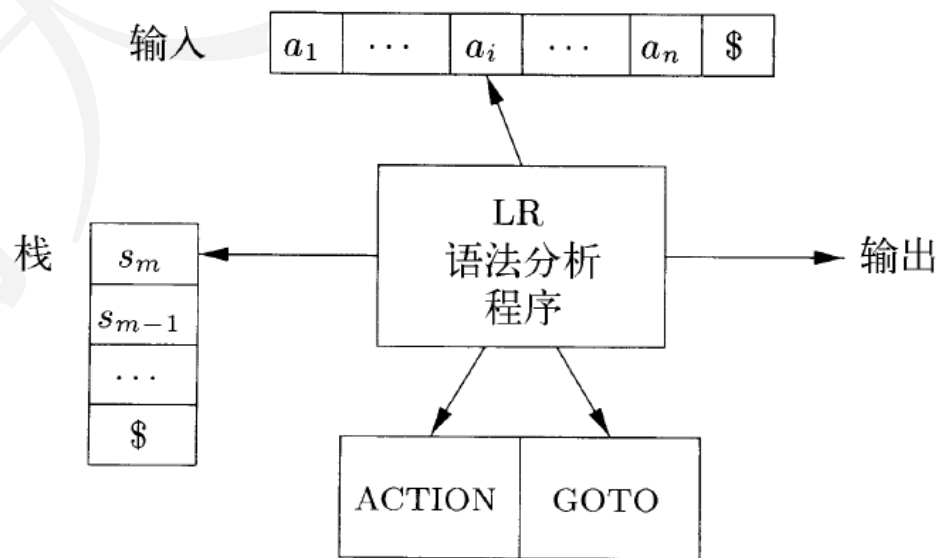
- 分析 $\text{id} * \text{id}$
 - 栈中只保留状态，文法符号可以从相应的状态中获取

行号	栈	符号	输入	动作
(1)	0	\$	id * id \$	移入到 5
(2)	0 5	\$ id	* id \$	按照 $F \rightarrow \text{id}$ 归约
(3)	0 3	\$ F	* id \$	按照 $T \rightarrow F$ 归约
(4)	0 2	\$ T	* id \$	移入到 7
(5)	0 2 7	\$ T *	id \$	移入到 5
(6)	0 2 7 5	\$ T * id	\$	按照 $F \rightarrow \text{id}$ 归约
(7)	0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
(8)	0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
(9)	0 1	\$ E	\$	接受

非必需

LR语法分析器的结构

- 所有的分析器都使用相同的驱动程序
- 分析表随文法以及LR分析技术的不同而不同
- 栈中存放的是状态序列，可求出相应的符号序列
- 分析程序根据栈顶状态和当前输入，通过分析表确定下一步动作



LR语法分析表的结构

- 两个部分：动作ACTION、转换GOTO
- ACTION表项有两个参数：状态 i ，终结符号 a
 - 移入 j ： j 是新状态，把 j 压入栈 (同时移入 a)
 - 归约 $A \rightarrow \beta$ ：把栈顶的 β 归约为 A (并根据GOTO表项压入新状态)
 - 接受：接受输入，完成分析
 - 报错：在输入中发现语法错误
- GOTO表项
 - 如果 $\text{GOTO}[I_i, A] = I_j$ ，那么 $\text{GOTO}[i, A] = j$

GOTO函数

GOTO表项

LR语法分析器的格局

- LR语法分析器的格局 (configuration) 包含了栈中内容和余下输入 ($s_0s_1\dots s_m, a_ia_{i+1}\dots a_n\$$)
 - 第一个分量是栈中的内容 (右侧是栈顶)
 - 第二个分量是余下的输入
- LR语法分析器的每一个状态都对应一个文法符号(s_0 除外)
 - 如果进入状态 s 的边的标号为符号 X , 那么 s 就对应于 X
 - 令 X_i 为 s_i 对应的符号, 那么 $X_1X_2\dots X_ma_ia_{i+1}\dots a_n$ 对应于一个最右句型

LR语法分析器的行为

- 对于格局 $(s_0s_1\ldots s_m, a_i a_{i+1}\ldots a_n \$)$, LR语法分析器查询条目 $\text{ACTION}[s_m, a_i]$ 确定相应的动作
 - 移入 s : 执行移入动作, 将状态 s (对应输入 a_i) 移入栈中, 得到新格局 $(s_0s_1\ldots s_m s, a_{i+1}\ldots a_n \$)$
 - 归约 $A \rightarrow \beta$: 将栈顶的 β 归约为 A , 压入状态 s , 得到新格局 $(s_0s_1\ldots s_{m-r} s, a_i a_{i+1}\ldots a_n \$)$, 其中 r 是 β 的长度, 状态 $s = \text{GOTO}[s_{m-r}, A]$
 - 接受: 语法分析过程完成
 - 报错: 发现语法错误, 调用错误恢复例程

LR语法分析算法

- 输入：文法 G 的LR语法分析表，输入串 w
- 输出：如果 w 在 $L(G)$ 中，则输出自底向上语法分析过程中的归约步骤，否则输出错误指示

- 算法：

```
令  $a$  为  $w\$$  的第一个符号；
while(1) { /* 永远重复 */
    令  $s$  是栈顶的状态；
    if ( ACTION[ $s, a$ ] = 移入  $t$  ) {
        将  $t$  压入栈中；
        令  $a$  为下一个输入符号；
    } else if ( ACTION[ $s, a$ ] = 归约  $A \rightarrow \beta$  ) {
        从栈中弹出 $|\beta|$ 个符号；
        令  $t$  为当前的栈顶状态；
        将 GOTO[ $t, A$ ] 压入栈中；
        输出产生式  $A \rightarrow \beta$ ；
    } else if ( ACTION[ $s, a$ ] = 接受 ) break; /* 语法分析完成 */
    else 调用错误恢复例程；
}
```

LR分析表的例子

- 文法：

(1) $E \rightarrow E + T$	(2) $E \rightarrow T$
(3) $T \rightarrow T * F$	(4) $T \rightarrow F$
(5) $F \rightarrow (E)$	(6) $F \rightarrow \text{id}$

状态	ACTION						GOTO		
	id	+	*	(	)	\$	<i>E</i>	<i>T</i>	<i>F</i>
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR分析过程的例子

- 输入: $\text{id} * \text{id} + \text{id}$

	栈	符号	输入	动作
(1)	0		id * id + id \$	移入
(2)	0 5	id	* id + id \$	根据 $F \rightarrow \text{id}$ 归约
(3)	0 3	F	* id + id \$	根据 $T \rightarrow F$ 归约
(4)	0 2	T	* id + id \$	移入
(5)	0 2 7	T *	id + id \$	移入
(6)	0 2 7 5	T * id	+ id \$	根据 $F \rightarrow \text{id}$ 归约
(7)	0 2 7 10	T * F	+ id \$	根据 $T \rightarrow T * F$ 归约
(8)	0 2	T	+ id \$	根据 $E \rightarrow T$ 归约
(9)	0 1	E	+ id \$	移入
(10)	0 1 6	E +	id \$	移入
(11)	0 1 6 5	E + id	\$	根据 $F \rightarrow \text{id}$ 归约
(12)	0 1 6 3	E + F	\$	根据 $T \rightarrow F$ 归约
(13)	0 1 6 9	E + T	\$	根据 $E \rightarrow E + T$ 归约
(14)	0 1	E	\$	接受

SLR语法分析表的构造

- 以LR(0)自动机为基础的SLR语法分析表构造算法
 - 构造增广文法 G' 的LR(0)项集规范族 $\{ I_0, I_1, \dots, I_n \}$
 - 状态 i 对应项集 I_i , 相关的ACTION/GOTO表条目如下
 - $[A \rightarrow \alpha \cdot a \beta]$ 在 I_i 中, 且 $\text{GOTO}(I_i, a) = I_j$, 则 $\text{ACTION}[i, a] = \text{"移入 } j\text{"}$
 - $[A \rightarrow \alpha \cdot]$ 在 I_i 中, 那么对 $\text{FOLLOW}(A)$ 中所有 a , $\text{ACTION}[i, a] = \text{"按 } A \rightarrow \alpha \text{ 归约"}$
 - 如果 $[S' \rightarrow S \cdot]$ 在 I_i 中, 那么将 $\text{ACTION}[i, \$]$ 设为 "接受"
 - 如果 $\text{GOTO}(I_i, A) = I_j$, 那么在GOTO表中, $\text{GOTO}[i, A] = j$
 - 空白的条目设为 "error"
- 如果SLR分析表没有冲突, 该文法就是SLR的

思想: 把 α 归约成为 A , 后面需是 $\text{FOLLOW}(A)$ 中的终结符号, 否则只能移入

SLR分析表构造的例子

- 项集 I_0 :

$E' \rightarrow \cdot E$

$E \rightarrow \cdot E+T$

$E \rightarrow \cdot T$

$T \rightarrow \cdot T^*F$

$T \rightarrow \cdot F$

$F \rightarrow \cdot (E)$

$F \rightarrow \cdot \text{id}$

- ACTION[0, (] = s4 ACTION[0, id] = s5

- GOTO[0, E] = 1 GOTO[0, T] = 2 GOTO[0, F] = 3

- 项集 I_1 : $E' \rightarrow E \cdot$ $E \rightarrow E \cdot +T$

- ACTION[1, +] = s6 ACTION[1, \$] = 接受

- 项集 I_2 : $E \rightarrow T \cdot$ $T \rightarrow T \cdot ^*F$

- ACTION[2, *] = s7 ACTION[2, +/)/\$] = 归约 $E \rightarrow T$

非SLR(1)文法的例子

- 文法
 - $S \rightarrow L = R \mid R$
 - $L \rightarrow *R \mid \text{id}$
 - $R \rightarrow L$
- 对于项集 I_2
 - 第一个项使 $\text{ACTION}[2, =] = \text{s6}$
 - 第二个项使 $\text{ACTION}[2, =] = \text{归约 } R \rightarrow L$

$I_0:$	$S' \rightarrow \cdot S$ $S \rightarrow \cdot L = R$ $S \rightarrow \cdot R$ $L \rightarrow \cdot * R$ $L \rightarrow \cdot \text{id}$ $R \rightarrow \cdot L$	$I_5:$	$L \rightarrow \text{id} \cdot$
$I_1:$	$S' \rightarrow S \cdot$	$I_6:$	$S \rightarrow L = \cdot R$ $R \rightarrow \cdot L$ $L \rightarrow \cdot * R$ $L \rightarrow \cdot \text{id}$
$I_2:$	$S \rightarrow L \cdot = R$ $R \rightarrow L \cdot$	$I_7:$	$L \rightarrow * R \cdot$
$I_3:$	$S \rightarrow R \cdot$	$I_8:$	$R \rightarrow L \cdot$
$I_4:$	$L \rightarrow * \cdot R$ $R \rightarrow \cdot L$ $L \rightarrow \cdot * R$ $L \rightarrow \cdot \text{id}$	$I_9:$	$S \rightarrow L = R \cdot$

可行前缀 (1)

- LR(0)自动机刻画了可能出现在移入-归约语法分析栈中的文法符号串
- 可行前缀 (viable prefix)
 - 可以出现在语法分析器栈中的最右句型的前缀，且没有越过该句型的句柄的右端
- 有效项
 - 如果存在一个推导过程 S 到 $\alpha Aw \Rightarrow \alpha \beta_1 \beta_2 w$ ，那么我们说项 $A \rightarrow \beta_1 \beta_2$ 是可行前缀 $\alpha \beta_1$ 的有效项

可行前缀 (2)

- 如果我们知道项 $A \rightarrow \beta_1 \cdot \beta_2$ 对 $\alpha\beta_1$ 有效
 - 当 β_2 不等于空，表示句柄尚未出现在栈中，应该移入
 - 如果 β_2 等于空，表示句柄已出现在栈中，应该归约
- 如果在某个时刻存在两个有效项要求对同一个可行前缀执行不同的动作，就应该设法解决冲突
 - 冲突实际上表示该可行前缀可能是两个最右句型的前缀，第一个包含了句柄，而另一个尚未包含句柄
 - 也可能都认为包含句柄，但是规则不一样

可行前缀 (3)

- SLR解决冲突的思想
 - 假如要按照 $A \rightarrow \beta$ 进行归约，只有当下一个输入符号在 $\text{FOLLOW}(A)$ 中时才可以归约
 - 不是所有的冲突都能通过查看下一个输入符号来解决

SLR语法分析器的弱点 (1)

- SLR技术解决冲突的方法
 - 项集中包含 $[A \rightarrow \alpha \cdot]$ 时，按照 $A \rightarrow \alpha$ 进行归约的条件是下一个输入符号 a 可以在某个句型中跟在 A 之后
 - 如果此时对于 a 还有其它的移入/归约操作，则出现冲突
 - 假设此时栈中的符号串为 $\beta\alpha$
 - 如果 $\beta A a$ 不是任何最右句型的前缀，那么即使 a 在某个句型中跟在 A 之后，仍不应该按 $A \rightarrow \alpha$ 归约
 - 进行归约的条件更加严格可以降低冲突的可能性

SLR语法分析器的弱点 (2)

- $[A \rightarrow \alpha \cdot]$ 出现在项集中的条件
 - 首先 $[A \rightarrow \cdot \alpha]$ 出现在某个项集中，然后逐步读入/归约到 α 中的符号，点不断后移，到达末端
 - 而 $[A \rightarrow \cdot \alpha]$ 出现的条件是 $B \rightarrow \beta \cdot A \gamma$ 出现在项中
 - 期望首先按照 $A \rightarrow \alpha$ 归约，然后将 $B \rightarrow \beta \cdot A \gamma$ 中的点移到 A 之后
 - 显然，在按照 $A \rightarrow \alpha$ 归约时要求下一个输入符号是 γ 的第一个符号，但是从 LR(0) 项集中不能确定这个信息

更强大的LR语法分析器

- 规范LR方法 (LR方法)
 - 添加项 $[A \rightarrow \cdot \alpha]$ 时, 把期望的向前看符号也加入项中 (成为LR(1)项集)
 - 这个做法可以充分利用向前看符号, 但是状态很多
- 向前看LR (LALR方法)
 - 基于LR(0)项集族, 但每个LR(0)项都带有向前看符号
 - 分析能力强于SLR方法, 且分析表和SLR分析表一样大
 - LALR已经可以处理大部分的程序设计语言

LR(1)项

- LR(1)项中包含更多信息来消除一些归约动作
- 实际的做法相当于“分裂”一些LR(0)状态，精确指明何时应该归约
- LR(1)项的形式 $[A \rightarrow \alpha \cdot \beta, a]$
 - a 称为向前看符号，可以是终结符号或者 $\$$
 - a 表示如果将来要按照 $A \rightarrow \alpha \beta$ 进行归约，归约时的下一个输入符号必须是 a
 - 当 β 非空时，移入动作不考虑 a ， a 传递到下一状态

LR(1)项和可行前缀

- $[A \rightarrow \alpha \cdot \beta, a]$ 对一个可行前缀 γ 有效的条件
 - 存在一个推导 $S \xRightarrow{*}_{rm} \delta A w \xRightarrow{rm} \delta \alpha \beta w$
 - 其中 $\gamma = \delta \alpha$, 且 a 是 w 的第一个符号, 或 w 为空且 $a = \$$
- 如果 $[A \rightarrow \alpha \cdot B \beta, a]$ 对于可行前缀 γ 有效, 那么
 - $[B \rightarrow \cdot \theta, b]$ 对于 γ 有效的条件是什么?
 - $S \xRightarrow{*}_{rm} \delta A w \xRightarrow{rm} \delta \alpha B \beta w \xRightarrow{*}_{rm} \delta \alpha B x w \xRightarrow{rm} \delta \alpha \theta x w$
 - b 应该是 xw 的第一个符号, 或 xw 为空且 $b = \$$
 - 如果 x 为空, 则 $b = a$
- 如果 $[A \rightarrow \alpha \cdot X \beta, a]$ 对可行前缀 γ 有效, 那么 $[A \rightarrow \alpha X \cdot \beta, a]$ 对 γX 有效

构造LR(1)项集

- LR(1)项集族的构造和LR(0)项集族类似，但是CLOSURE和GOTO有所不同
 - 在CLOSURE中，当由项 $[A \rightarrow \alpha \cdot B\beta, a]$ 生成新项 $[B \rightarrow \cdot \theta, b]$ 时， b 必须在 $\text{FIRST}(\beta a)$ 中
 - 对LR(1)项集中的任意项 $[A \rightarrow \alpha \cdot B\beta, a]$ ，总有： a 在 $\text{FOLLOW}(A)$ 中
 - 初始项满足这个条件
 - 每次求CLOSURE项集时，新产生的项也满足这个条件

LR(1)项集的CLOSURE算法

- 注意在添加 $[B \rightarrow \cdot\gamma, b]$ 时, b 的取值范围

```
SetOfItems CLOSURE( $I$ ) {  
    repeat  
        for (  $I$  中的每个项  $[A \rightarrow \alpha \cdot B \beta, a]$  )  
            for (  $G'$  中的每个产生式  $B \rightarrow \gamma$  )  
                for ( FIRST( $\beta a$ ) 中的每个终结符号  $b$  )  
                    将  $[B \rightarrow \cdot\gamma, b]$  加入到集合  $I$  中;  
    until 不能向  $I$  中加入更多的项;  
    return  $I$ ;  
}
```

LR(1)项集的GOTO算法

- 和LR(0)项集的GOTO算法基本相同

```
SetOfItems GOTO( $I, X$ ) {  
    将  $J$  初始化为空集;  
    for ( $I$  中的每个项  $[A \rightarrow \alpha \cdot X \beta, a]$ )  
        将项  $[A \rightarrow \alpha X \cdot \beta, a]$  加入到集合  $J$  中;  
    return CLOSURE( $J$ );  
}
```

LR(1)项集族的构造算法

- 和LR(0)项集族的构造算法相同

课本有误，
此处修正

```
void items( $G'$ ) {  
    将  $C$  初始化为  $\{ \text{CLOSURE}(\{[S' \rightarrow \cdot S, \$]\}) \}$   
    repeat  
        for (  $C$  中的每个项集  $I$  )  
            for ( 每个文法符号  $X$  )  
                if (  $\text{GOTO}(I, X)$  非空且不在  $C$  中 )  
                    将  $\text{GOTO}(I, X)$  加入  $C$  中;  
    until 不再有新的项集加入到  $C$  中;  
}
```

例子 (1)

- 增广文法

- $S' \rightarrow S \quad S \rightarrow CC \quad C \rightarrow cC \mid d$

- 构造 I_0 项集和GOTO函数

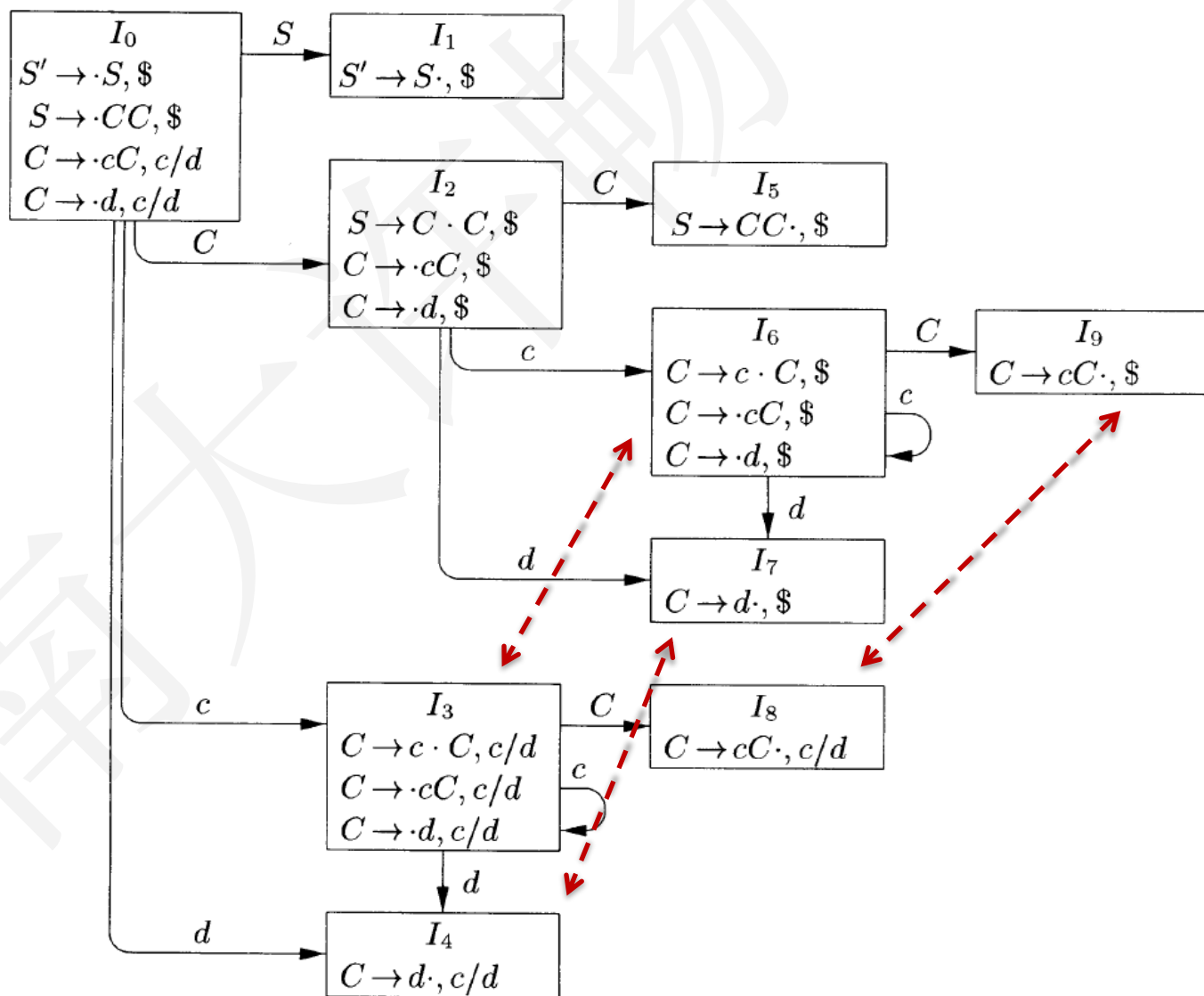
- $I_0 = \text{CLOSURE}\{ [S' \rightarrow \cdot S, \$] \} =$
 - $\{ [S' \rightarrow \cdot S, \$], [S \rightarrow \cdot CC, \$], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d] \}$
 - $\text{GOTO}(I_0, S) = \{ [S' \rightarrow S\cdot, \$] \}$
 - $\text{GOTO}(I_0, C) = \text{CLOSURE}\{ [S \rightarrow C\cdot C, \$] \} =$
 - $\{ [S \rightarrow C\cdot C, \$], [C \rightarrow \cdot cC, \$], [C \rightarrow \cdot d, \$] \}$
 - $\text{GOTO}(I_0, c) = \text{CLOSURE}\{ [C \rightarrow c\cdot C, c/d] \} =$
 - $\{ [C \rightarrow c\cdot C, c/d], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d] \}$
 - $\text{GOTO}(I_0, d) = \{ [C \rightarrow d\cdot, c/d] \}$

例子 (2)

不计向前看符号

- I_3, I_6 相同
- I_4, I_7 相同
- I_8, I_9 相同

共10个状态



LR(1)语法分析表的构造

- 步骤
 - 构造得到LR(1)项集族 $C' = \{ I_0, I_1, \dots, I_n \}$
 - 状态 i 对应于项集 I_i , 其分析动作如下
 - $[A \rightarrow \alpha \cdot a\beta, b]$ 在项集中, 且 $\text{GOTO}(I_i, a) = I_j$, 那么 $\text{ACTION}[i, a] = \text{"移入 } j\text{"}$
 - $[A \rightarrow \alpha \cdot, a]$ 在项集中, 那么 $\text{ACTION}[i, a] = \text{"按照 } A \rightarrow \alpha \text{ 归约"}$
 - $[S' \rightarrow S \cdot, \$]$ 在项集中, 那么 $\text{ACTION}[i, \$] = \text{"接受"}$
 - GOTO表项: $\text{GOTO}[i, A] = j$, 如果 $\text{GOTO}(I_i, A) = I_j$
 - 没有填写的条目为error
 - 如果条目有冲突, 则不是LR(1)的
 - 初始状态对应于 $[S' \rightarrow \cdot S, \$]$ 所在的项集

例子

- (3, 6), (4, 7), (8, 9)可看作是由原来的一个LR(0)状态拆分而来

文法：

$S' \rightarrow S$

$S \rightarrow C C$

$C \rightarrow c C \mid d$

状态	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

构造LALR语法分析表

- SLR(1)语法分析表的分析能力较弱
- LR(1)语法分析表的状态数量很大
- LALR(1)是实践中常用的方法
 - 状态数量和SLR(1)的状态数量相同
 - 能够方便地处理大部分常见程序设计语言的构造

LR(1)语法分析表的合并

- 4和7在向前看符号上不同
 - $[C \rightarrow d\cdot, c/d]$ vs. $[C \rightarrow d\cdot, \$]$
 - 状态4: 下个符号为c/d则归约, 为\$则报错
 - 状态7: 分析动作正好相反
- 如果将4和7中的项合并得47, 则所有情况下都归约
 - 对这个文法, 合并不会引起冲突, 但有些文法会

状态	ACTION			GOTO	
	<i>c</i>	<i>d</i>	<i>\$</i>	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

文法:

$S' \rightarrow S$

$S \rightarrow C C$

$C \rightarrow c C \mid d$

LALR分析技术的基本思想

- 寻找具有相同核心的LR(1)项集，并把它们合并成为一个项集
 - 项集的核心 (core) 就是项的第一分量的集合
 - I_4 和 I_7 的核心都是 $\{C \rightarrow d \cdot\}$
 - I_3 和 I_6 的核心 $\{C \rightarrow c \cdot C, C \rightarrow \cdot cC, C \rightarrow \cdot d\}$
- 一个LR(1)项集的核心是一个LR(0)项集
- $GOTO(I, X)$ 的核心只由 I 的核心决定，因此被合并项集的GOTO目标也可以合并
 - 这表示合并之后，我们仍可以建立GOTO关系

合并引起的冲突

- 原来无冲突的LR(1)分析表在合并之后得到LALR(1)分析表，新表中可能存在冲突
- 合并不会导致移入/归约冲突
 - 假设合并之后在 a 上存在移入/归约冲突，即存在项 $[B \rightarrow \beta \cdot a \gamma, ?]$ 和 $[A \rightarrow \alpha \cdot, a]$
 - 因为被合并的项集具有相同的核心，因此被合并的所有项集中都包括 $[B \rightarrow \beta \cdot a \gamma, ?]$ ，而 $[A \rightarrow \alpha \cdot, a]$ 也必然在某个项集中，那么这个项集必然已经存在冲突！
- 合并会引起归约/归约冲突，即不能确定按照哪个产生式进行归约

合并引起归约/归约冲突的例子

- 文法：
 - $S' \rightarrow S$ $S \rightarrow a A d \mid b B d \mid a B e \mid b A e$
 - $A \rightarrow c$ $B \rightarrow c$
- 语言：{ acd, bcd, ace, bce }
- 可行前缀ac的有效项集：{ $[A \rightarrow c\cdot, d]$, $[B \rightarrow c\cdot, e]$ }
- 可行前缀bc的有效项集：{ $[B \rightarrow c\cdot, d]$, $[A \rightarrow c\cdot, e]$ }
- 合并之后的项集为：{ $[A \rightarrow c\cdot, d/e]$, $[B \rightarrow c\cdot, d/e]$ }
 - 包含了归约/归约冲突：应该把c归约成为A还是B?

LALR分析表构造算法

- 步骤
 - 构造得到LR(1)项集族 $C = \{ I_0, I_1, \dots, I_n \}$
 - 对于LR(1)中的每个核心，找出所有具有该核心的项集，并把这些项集替换为它们的并集
 - 令 $C' = \{ J_0, J_1, \dots, J_m \}$ 为得到的LR(1)项集族
 - 按照LR分析表的构造方法得到ACTION表 (如果存在冲突，则这个文法不是LALR的)
 - GOTO表的构造：设 J 是一个或者多个LR(1)项集 (包括 I_1) 的并集，令 K 是所有和 $\text{GOTO}(I_1, X)$ 具有相同核心的项集的并集，那么 $\text{GOTO}(J, X) = K$
- 得到的分析表称为**LALR语法分析表**

例子

- LR(1)项集族中有三对可以合并
 - $I_{36}: [C \rightarrow c \cdot C, c/d/\$, [C \rightarrow \cdot cC, c/d/\$, [C \rightarrow \cdot d, c/d/\$]$
 - $I_{47}: [C \rightarrow d \cdot, c/d/\$]$
 - $I_{89}: [C \rightarrow cC \cdot, c/d/\$]$
- $GOTO(I_{36}, C) = I_{89}$ (原 $GOTO(I_3, C) = I_8$)

文法:

$S' \rightarrow S$

$S \rightarrow C C$

$C \rightarrow c C \mid d$

状态	ACTION			GOTO	
	<i>c</i>	<i>d</i>	<i>\$</i>	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7		5	
3	s3	s4		8	
4	r3	r3			
5			r1		
6	s6	s7		9	
7			r3		
8	r2	r2			
9			r2		



状态	ACTION			GOTO	
	<i>c</i>	<i>d</i>	<i>\$</i>	<i>S</i>	<i>C</i>
0	s36	s47		1	2
1			acc		
2	s36	s47		5	
36	s36	s47		89	
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

LALR分析器和LR分析器

- 处理语法正确的输入时，LALR语法分析器和LR语法分析器的动作序列完全相同
 - 栈中的状态名字不同，但是状态序列之间有对应关系
 - 如果LR分析器压入状态 I ，那么LALR分析器压入 I 对应的合并项集
 - 当LR分析器压入状态 I_3 时，LALR分析器压入状态 I_{36}
- 当处理错误的输入时，LALR可能多执行一些归约动作，但不会多移入一个符号

LALR技术本质

- 对LR(1)项集规范族中的同核心项集进行合并
 - 使得分析表保持了LR(1)项中的向前看符号信息
 - 又使状态数减少到与SLR分析表的一样多

二义性文法的使用

- 二义性文法都不是LR的
- 某些二义性文法是有用的
 - 可以简洁地描述某些结构
 - 隔离某些语法结构，对其进行特殊处理
- 对于某些二义性文法
 - 可以通过消除二义性规则来保证每个句子只有一棵语法分析树
 - 可以在LR分析器中实现这个规则

优先级/结合性消除冲突

- 二义性文法

- $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$

- 等价于

- $E \rightarrow E + T \mid T \qquad T \rightarrow T * F \mid F \qquad F \rightarrow (E) \mid \text{id}$

- 二义性文法的优点

- 容易修改算符的**优先级**和**结合性**

- **简洁**：如果有多个优先级，那么无二义性文法将引入太多的非终结符号

- **高效**：不需要处理像 $E \rightarrow T$ 这样的归约

二义性表达式文法的LR(0)项集

- 文法

- $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$

- 冲突

- I_7 、 I_8 中有冲突，且不可能通过向前看符号解决！

I_0 : $E' \rightarrow \cdot E$
 $E \rightarrow \cdot E + E$
 $E \rightarrow \cdot E * E$
 $E \rightarrow \cdot (E)$
 $E \rightarrow \cdot \text{id}$

I_1 : $E' \rightarrow E \cdot$
 $E \rightarrow E \cdot + E$
 $E \rightarrow E \cdot * E$

I_2 : $E \rightarrow (\cdot E)$
 $E \rightarrow \cdot E + E$
 $E \rightarrow \cdot E * E$
 $E \rightarrow \cdot (E)$
 $E \rightarrow \cdot \text{id}$

I_3 : $E \rightarrow \text{id} \cdot$

I_4 : $E \rightarrow E + \cdot E$
 $E \rightarrow \cdot E + E$
 $E \rightarrow \cdot E * E$
 $E \rightarrow \cdot (E)$
 $E \rightarrow \cdot \text{id}$

I_5 : $E \rightarrow E * \cdot E$
 $E \rightarrow \cdot E + E$
 $E \rightarrow \cdot E * E$
 $E \rightarrow \cdot (E)$
 $E \rightarrow \cdot \text{id}$

I_6 : $E \rightarrow (E \cdot)$
 $E \rightarrow E \cdot + E$
 $E \rightarrow E \cdot * E$

I_7 : $E \rightarrow E + E \cdot$
 $E \rightarrow E \cdot + E$
 $E \rightarrow E \cdot * E$

I_8 : $E \rightarrow E * E \cdot$
 $E \rightarrow E \cdot + E$
 $E \rightarrow E \cdot * E$

I_9 : $E \rightarrow (E) \cdot$

冲突的原因以及解决

- 当栈顶状态为7时，表明
 - 栈中状态序列对应的文法符号序列为：... $E + E$
 - 如果下一个符号为+或*，移入还是归约？
- 如果*的优先级大于+，且+是左结合的
 - 下一个符号为*时，我们应该移入*
 - 下一个符号为+时，我们应该将 $E + E$ 归约为 E
- 栈顶状态为8时，有类似情况

解决冲突之后的SLR(1)分析表

- 对于状态7
 - +时归约
 - *时移入
- 对于状态8
 - 总执行归约

状态	ACTION						GOTO
	id	+	*	(	)	\$	E
0	s3			s2			1
1		s4	s5			acc	
2	s3			s2			6
3		r4	r4		r4	r4	
4	s3			s2			7
5	s3			s2			8
6		s4	s5		s9		
7		r1	s5		r1	r1	
8		r2	r2		r2	r2	
9		r3	r3		r3	r3	

- 这个表和等价的无二义性文法的分析表类似

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

悬空else的二义性

- 文法

$$S' \rightarrow S$$

$$S \rightarrow iSeS \mid iS \mid a$$

- 项集 I_4 包含冲突

- 栈中符号为： iS

- 冲突

- 下一个符号为 e ，因栈中 i 未匹配，该移入 e
- 如果下一个符号属于 $FOLLOW(S)$ ，该归约
- 解决：移入

$I_0:$ $S' \rightarrow \cdot S$
 $S \rightarrow \cdot iSeS$
 $S \rightarrow \cdot iS$
 $S \rightarrow \cdot a$

$I_1:$ $S' \rightarrow S \cdot$

$I_2:$ $S \rightarrow i \cdot SeS$
 $S \rightarrow i \cdot S$
 $S \rightarrow i \cdot SeS$
 $S \rightarrow i \cdot S$
 $S \rightarrow i \cdot a$

$I_3:$ $S \rightarrow a \cdot$

$I_4:$ $S \rightarrow iS \cdot eS$
 $S \rightarrow iS \cdot$

$I_5:$ $S \rightarrow iSe \cdot S$
 $S \rightarrow \cdot iSeS$
 $S \rightarrow \cdot iS$
 $S \rightarrow \cdot a$

$I_6:$ $S \rightarrow iSeS \cdot$

语法错误的处理

- 错误难以避免，编译器需要有处理错误的能力
- 程序中可能存在不同层次的错误
 - 词法错误、语法错误、语义错误、逻辑错误
- 语法分析器中错误处理程序的设计目标
 - 清晰准确地报告出现的错误，并指出错误的**位置**
 - 能从当前错误中**恢复**，以继续检测后面的错误
 - 尽可能地减少开销

预测分析中的错误恢复

- 错误恢复
 - 当预测分析器报错时，表示输入的串不是句子
 - 使用者希望预测分析器能够进行恢复处理后继续语法分析过程，以便在一次分析中找到更多的语法错误
 - 可能恢复得并不成功，之后找到的语法错误是假的
 - 进行错误恢复时可用的信息：栈里面的符号、待分析的符号
- 两类错误恢复方法
 - 恐慌模式、短语层次的恢复

恐慌模式

- 基本思想
 - 语法分析器忽略输入中的一些符号，直到出现由设计者选定的某个同步词法单元
 - 解释
 - 语法分析过程总是试图在输入前缀中找到和某个非终结符号对应的串，出现错误表明已经不可能找到对应这个非终结符号(程序结构)的串了
 - 如果编程错误仅局限于这个程序结构，我们可考虑跳过该程序结构，假装已经找到了它，然后继续进行语法分析处理
 - 同步词法单元(synchronizing token)就是这个程序结构结束的标志

同步词法单元的确定

- 文法符号 A 的同步集合的启发式规则

非
终
结
符

- 将 $\text{FOLLOW}(A)$ 中所有符号放入 A 的同步集合中
- 将高层次非终结符号对应串的开始符号加入到较低层次非终结符号的同步集合
 - 比如：语句的开始符号，if/while等可作为表达式的同步集合
- 将 $\text{FIRST}(A)$ 中的符号加入到 A 的同步集合
 - 碰到这些符号时，可能意味着前面的符号是多余的符号
- 如果 A 可以推导出空串，把该产生式当作默认值

终
结
符

- 在栈顶的终结符号出现匹配错误时，可直接弹出该符号，并且发出消息称已经插入了这个终结符号

- 根据特定应用来决定哪些符号需要确定同步集合

恐慌模式的例子 (1)

- 对 E, T, F 的表达式进行语法分析时，可直接使用其 FOLLOW 值作为同步集合
 - synch 表示一直忽略到同步集合，然后弹出非终结符号
 - 空条目表示忽略输入符号 (多)
 - 终结符号不匹配时，弹出栈中终结符号 (漏)

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E \rightarrow +TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$	synch	synch	$F \rightarrow (E)$	synch	synch

恐慌模式的例子 (2)

- 错误输入
 - + id * + id

$$\text{FOLLOW}(F) = \{ *, +, \$,) \}$$

栈	输入	说明
$E \$$	+ id * + id \$	错误, 略过 +
$E \$$	id * + id \$	id 在 $\text{FIRST}(E)$ 中
$TE' \$$	id * + id \$	
$FT'E' \$$	id * + id \$	
id $T'E' \$$	id * + id \$	
$T'E' \$$	* + id \$	
* $FT'E' \$$	* + id \$	
$FT'E' \$$	+ id \$	错误, $M[F, +] = \text{synch}$
$T'E' \$$	+ id \$	F 已经被弹出栈
$E' \$$	+ id \$	
+ $TE' \$$	+ id \$	
$TE' \$$	id \$	
$FT'E' \$$	id \$	
id $T'E' \$$	id \$	
$T'E' \$$	\$	
$E' \$$	\$	
\$	\$	

短语层次的恢复

- 在预测语法分析表的空白条目中插入错误处理例程的函数指针
 - 例程可以改变、插入或删除输入中的符号，并发出适当的错误消息

LR语法分析中的错误恢复 (1)

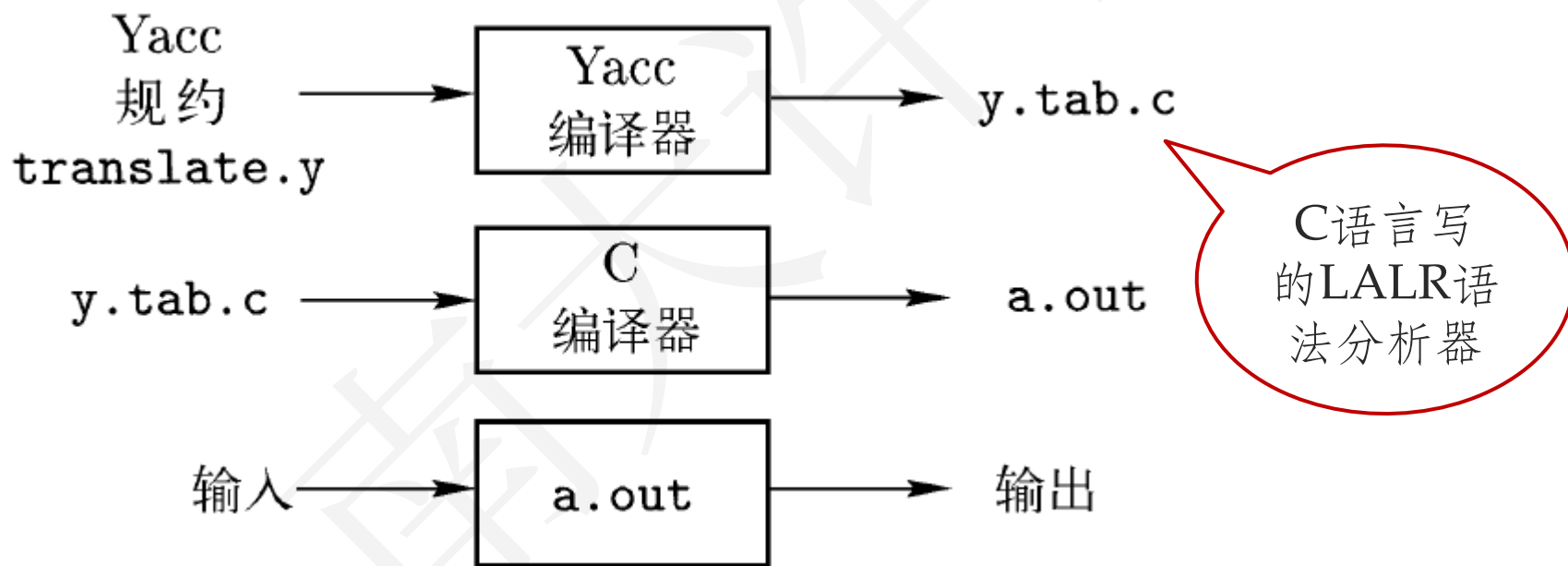
- 查询ACTION表时可能发现报错条目
 - 假设栈中的符号串为 α ，当前输入符号为 a ，报错表示不可能存在终结符号串 x 使得 αax 是一个最右句型
- 恐慌模式的错误恢复策略
 - 从栈顶向下扫描，找到状态 s ， s 有一个对应于某个非终结符号 A 的GOTO目标 (s 之上的状态被丢弃)
 - 在输入中丢弃一些符号，直到一个可以跟在 A 之后的符号 b (不丢弃 b)，并将GOTO(s, A)压栈，继续进行分析
 - 基本思想：假定当前试图归约到 A 但碰到了语法错误，因此设法扫描完包含语法错误的 A 的子串，假装找到了 A 的一个实例

LR语法分析中的错误恢复 (2)

- 短语层次的恢复
 - 检查LR分析表中的每个报错条目，根据语言的特性来确定程序员最可能犯了什么错误，然后构造适当的恢复程序

语法分析器生成工具Yacc

- Yacc/Bison的使用方法如下



Yacc源程序的结构

- 声明
 - 放置C声明和对词法单元的声明
- 翻译规则
 - 指明产生式及相关的语义动作
- 辅助性C语言例程
 - 被直接拷贝到生成的C语言源程序中
 - 可在语义动作中调用
 - 包括yylex(), 这个函数返回词法单元, 可以由Lex生成

声明
%%
翻译规则
%%
辅助性C语言例程

翻译规则的格式

- 说明

- 第一个产生式的头被看作开始符号
 - 语义动作是C语句序列
 - $$$$ 表示和产生式头相关的属性值, $\$i$ 表示产生式体中第 i 个文法符号的属性值
 - 在按照某个产生式归约时, 执行相应的语义动作, 可以根据 $\$i$ 来计算 $$$$ 的值
- <产生式头>: <产生式体1> { <语义动作1> }
 | <产生式体2> { <语义动作2> }

 | <产生式体 n > { <语义动作 n > }
 ;

Yacc源程序的例子

```
%{
#include <ctype.h>
%}

%token DIGIT

%%
line  : expr '\n'      { printf("%d\n", $1); }
      ;
expr  : expr '+' term  { $$ = $1 + $3; }
      | term
      ;
term  : term '*' factor { $$ = $1 * $3; }
      | factor
      ;
factor: '(' expr ')'    { $$ = $2; }
      | DIGIT
      ;

%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}
```

Yacc中的冲突处理

- 缺省处理方法
 - 归约/移入冲突：总是移入 (悬空else的解决)
 - 归约/归约冲突：选择列在前面的产生式
 - 选项-v可在文件y.output中看到冲突的描述和解决方法
- 通过确定终结符号的优先级/结合性来解决冲突
 - 结合性：%left, %right, %nonassoc
 - 移入 a /按 $A \rightarrow \alpha$ 归约：比较 a 和 $A \rightarrow \alpha$ 的优先级再选择
 - 终结符号的优先级按在声明部分的出现顺序而定
 - 产生式的优先级设为它最右的终结符号的优先级，也可以加标记%prec<终结符号>，指明产生式的优先级等同于该终结符号

Yacc的错误恢复

- 使用错误产生式来完成语法错误恢复
 - 错误产生式 $A \rightarrow \text{error } \alpha$
 - 例如: $\text{stmt} \rightarrow \text{error};$
- 定义哪些非终结符号有错误恢复动作
 - 比如: 表达式、语句、块、函数定义等非终结符号
- 当语法分析器遇到错误时
 - 不断弹出栈中状态, 直到栈顶状态包含项 $A \rightarrow \cdot \text{error } \alpha$
 - 分析器将 **error** 移入栈中
 - 如果 α 为空, 分析器直接执行 **归约**, 并调用相关的语义动作; 否则 **跳过** 一些符号, 找到可以归约为 α 的串为止